

DOT PRODUCT ENGINE BASED NEUROMORPHIC HARDWARE FOR
CONTINUOUS-TIME BIOMEDICAL SIGNAL CLASSIFICATION

A Thesis
presented to
the Faculty of California Polytechnic State University,
San Luis Obispo

In Partial Fulfillment
of the Requirements for the Degree
Master of Science in Electrical Engineering

by
Sanjeev Srinivasan

June 2026

© 2026

Sanjeev Srinivasan

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE: Dot Product Engine based Neuromorphic
Hardware for Continuous-Time Biomedical
Signal Classification

AUTHOR: Sanjeev Srinivasan

DATE SUBMITTED: June 2026

COMMITTEE CHAIR: Nishith Chakraborty, Ph.D.
Assistant Professor of Electrical Engineering

COMMITTEE MEMBER: Andrew Danowitz, Ph.D.
Professor of Computer Engineering

COMMITTEE MEMBER: Mohammad Ghamari, Ph.D.
Assistant Professor of Electrical Engineering

ABSTRACT

Dot Product Engine based Neuromorphic Hardware for Continuous-Time Biomedical
Signal Classification
Sanjeev Srinivasan

Accurate diagnosis of pathological conditions from biomedical signals, such as electrocardiograms (ECGs) is often performed offline, making it time-consuming, costly, and inefficient, especially when abnormal patterns are rare and long-term monitoring generates large amounts of data. To address this, this work proposes a compact, scalable, and programmable neuromorphic system designed for real-time preliminary arrhythmia detection and classification using ECG signals, that can be extended to other biomedical signals. The proposed design processes ECG signals using a delta modulation-based spike encoder, followed by classification with a dot-product engine (DPE) based spiking neural network (SNN) processor and winner-take-all (WTA) circuit. The architecture supports online learning via spike-timing-dependent-plasticity (STDP) and includes a scan-chain programming circuit that allows processor parameters to be reconfigured during runtime.

The proposed system has been implemented on four FPGA platforms, demonstrating promising results in terms of resource utilization, power consumption, and operating frequency. The optimized design uses only 2.63% of available LUTs and 1.04% of available FFs and features a maximum operating frequency of 192.8 MHz with 20 mW of dynamic power consumption at 100 MHz, which linearly scales, on the low cost, entry-level Basys-3 board. This design was subsequently implemented in a 130nm CMOS process, taking up a total die area of 95,792.5 μm^2 and dynamic power consumption as low as 26.2 μW for a 1MHz clock. Finally, a software model was built utilizing `snnTorch` to train a spiking neural network with surrogate gradient backpropagation and evaluation on a hardware-faithful simulation of the proposed system. Tested on the MIT-BIH Arrhythmia Database, the system achieves classification across the four main AAMI heartbeat classes, with significant potential for improving multi-class arrhythmia sensitivity and detection. These results, alongside the compact hardware footprint, position this initial design – an efficient, scalable solution for real-time arrhythmia detection, adaptable to diverse patient profiles and other biomedical signals – as a foundation for future enhancements including increasing synapse weight bit-widths and accumulator precision, scaling the DPE array size, cascading multiple arrays to enable multi-layer feature extraction, and implementing per-neuron programmable thresholds while maintaining ultra-low-power operation.

Keywords: Neuromorphic, ECG, spiking neural networks, dot-product engine, delta modulation, FPGA, ASIC.

TABLE OF CONTENTS

	Page
LIST OF TABLES	viii
LIST OF FIGURES	ix
CHAPTER	
1. INTRODUCTION	1
2. BACKGROUND	5
2.1. Clinical Challenge: Cardiovascular Diseases.....	5
2.1.1. Global Impact.....	5
2.2. Electrocardiogram (ECG)	6
2.2.1. The Electrophysiology of the Heart	6
2.2.2. Arrhythmias and Detection	7
2.2.3. AAMI ECG Classification and MIT-BIH Dataset.....	9
2.3. The Need for Edge Intelligence	10
2.3.1. Wearable Cardiac Devices	10
2.3.2. Constraints of Deep Learning at the Edge	11
2.4. Shift from ANNs to SNNs	12
2.4.1. Biological Inspiration: Spiking Leaky-Integrate-and-Fire Neuron.....	13
2.4.2. Advantages of SNNs.....	14
2.5. Spike Encoding Schemes	15
2.5.1. Rate vs. Temporal Encoding.....	15
2.5.2. Delta Modulation	16
2.6. Theory of the Dot Product Engine (DPE).....	16
2.6.1. Event-Driven Dot Products and Gated Accumulation	17
2.7. Learning Dynamics in SNNs	19
2.7.1. Spike Timing Dependent Plasticity (STDP)	19
2.7.2. Competitive Learning – Winner-Take-All (WTA).....	20
2.8. Related Works.....	21
3. HARDWARE ARCHITECTURE & DESIGN IMPLEMENTATION.....	23
3.1. System Overview and Implementation	23

3.1.1. Processing Elements	24
3.1.1.1. Delta Modulator	24
3.1.1.2. Dot Product Engine based Spiking Neural Network.....	27
3.1.1.2.1. Synapse Unit	28
3.1.1.2.2. Accumulator / Compare and Fire	31
3.1.1.3. Winner-Take-All	34
3.1.2. Top-Level Modules	35
3.1.2.1. Neuron Array	36
3.1.2.2. Scan Chain Wrapper	39
4. HARDWARE OPTIMIZATIONS	42
4.1. Design Constraints vs. Optimization Goals	42
4.2. Hardware Logic Tradeoffs	43
4.3. Architectural Evolution.....	43
4.3.1. Initial 2x5 Neuron Array with SCW	43
4.3.2. Improving Design Accuracy and Metrics	47
4.3.3. Final Metric Improvements.....	51
4.3.4. Changes for RTL to GDSII Flow	54
5. FPGA AND ASIC IMPLEMENTATIONS	55
5.1. FPGA Implementation	55
5.1.1. Verification Methodology	55
5.1.2. Behavioral Simulation - Top-Level Testbench around SCW	56
5.1.3. FPGA Results.....	58
5.1.3.1. Frequency Analysis.....	59
5.1.3.2. Dynamic Power Analysis	60
5.1.3.3. Resource Utilization Analysis.....	61
5.2. ASIC Implementation	63
5.2.1. RTL to GDSII Flow	63
5.2.2. ASIC Results	65
6. SOFTWARE MODEL	68
6.1. snnTorch Introduction.....	68
6.1.1. LIF Neuron.....	68

6.1.2. Training SNNs and Surrogate Gradients	69
6.1.3. Loss Function	71
6.2. Model Architecture and Results	72
6.2.1. Baseline Implementation and Results	72
6.2.2. Weight Quantization	75
6.2.3. Class-Balanced Training	76
6.2.4. Evaluation of Improved Model	76
7. CONCLUSION	80
7.1. Future Work	81
REFERENCES	84
APPENDIX.....	95

LIST OF TABLES

Table	Page
4.1: First Accurate Design Metrics	48
4.2: Intermediate Accurate Design Metrics	50
4.3: Best FPGA Implementation Performance Metrics	52
5.1: Resource Utilization Analysis Across 4 FPGA Boards.....	62
5.2: ASIC Implementation Metrics.....	65

LIST OF FIGURES

Figure	Page
2.1: Cardiac Conduction System and ECG Waveform [11]	7
2.2: Relevant Measurements from ECG Waveform [13]	8
2.3: Number of Each ECG Class in MIT-BIH Dataset by AAMI Standard [18]	10
2.4: Neuron Model in (a) ANNs and (b) SNNs [28].....	13
2.5: LIF (Post-neuron) Computation in SNN Representation [33].....	14
2.6: Computing Principles of (a) Crossbar Array based DPE and (b) Fully Connected Synaptic Layers [42].....	18
2.7: STDP based Learning in SNNs [47].....	20
3.1: Hardware Architecture Block Diagram	24
3.2: Delta Modulator DUT Simulation Waveform	27
3.3: Synapse Unit DUT Simulation Waveform	30
3.4: Accumulator DUT Simulation Waveform.....	34
3.5: WTA DUT Simulation Waveform	35
3.6: Neuron Array DUT Simulation Waveform	38
3.7: Scan Chain Data Processing Simulation Waveform.....	41
3.8: Scan Chain Wrapper DUT Simulation Waveform	41
4.1: Initial Neuron Array Timing Diagram.....	45
4.2: Scan Chain Wrapper to System Parameter Timing Diagram	46
4.3: Second Iteration Neuron Array Timing Diagram	48
4.4: Third Iteration Neuron Array Timing Diagram.....	50
4.5: Fourth Iteration Neuron Array Timing Diagram	52
5.1: SCW Testbench Simulation Waveform.....	58
5.2: Timing Margin Analysis across 4 FPGA Boards	59
5.3: Dynamic Power Consumption Analysis across 4 FPGA Boards	61
5.4: ASIC Implementation Layout at 100MHz.....	66
6.1: Recurrent Representation of Spiking Neurons [56].....	70
6.2: Spike Non-differentiability and Surrogate Gradient Descent [58]	71
6.3: Baseline Implementation Confusion Matrix and Classwise Accuracy.....	75
6.4: Mapping from Trained Raw Weights to 6-bit Signed Scaled Weights	76
6.5: Improved Model without STDP - Confusion Matrix and Classwise Accuracy	77
6.6: Improved Model with STDP - Confusion Matrix and Classwise Accuracy.....	78
6.7: Improved Model with Per-Class Thresholds – Confusion Matrix and Classwise Accuracy	79

1. INTRODUCTION

Cardiovascular Diseases (CVDs) have remained the leading cause of death worldwide, with a projected trend to remain the main health challenge in upcoming years [1]. The early detection of arrhythmias, or irregularities, in the heartbeat is key in preventing severe cardiac events, such as strokes or myocardial infarctions [2]. Electrocardiograms (ECGs) are the gold standard for quick, safe, non-invasive cardiac assessment, providing real-time information about heart function based on the electrical signals of the heart [3]. The ECG wave can be analyzed to detect arrhythmias. However, traditional clinical diagnosis is often performed offline with outdated techniques that are inconvenient and possibly inaccessible. Furthermore, potential long-term monitoring of the ECG signal generates massive amounts of data that is time-consuming and costly for experts to review.

The recent emergence of wearable technologies, such as smart watches, armbands, and disposable ECG patches, has enabled the continuous and convenient monitoring of vital signals in everyday environments [4], [5]. These devices can capture ECG signals with high-quality; however, the primary challenge lies in the automated, real-time classification of the ECG signal at the edge. Most modern classification techniques require the use of deep learning (DL) methods, such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs). While these techniques can achieve high accuracy for these classification tasks, they are computationally expensive and memory-intensive [5], [6]. Since wearable devices are battery-powered and resource-constrained, deploying power-hungry artificial neural networks (ANNs) often leads to offloading data to the cloud. This raises privacy concerns, incurs high costs for data transmission, and depends on a stable network connection [7].

To bridge the gap between high-fidelity monitoring and low-power execution, a 3rd-generation machine learning (ML) framework, spiking neural networks (SNNs) have emerged as a promising and viable alternative. SNNs operate in an event-driven, sparsely active domain, where information is processed as discrete temporal events, or binary spikes [3], [8]. SNNs are inherently suited for temporal biomedical signal analysis like ECG, as it mirrors the sparse and event-driven nature of biological heart rhythms. By utilizing neuromorphic computing principles, where memory and processing are collocated, SNNs significantly reduce power consumption and computational overhead compared to traditional von Neumann architectures [3], [9].

Despite the potential of SNNs, many existing implementations struggle with the complexities of real-time signal encoding [3] and the hardware-level reconfigurability required to be adaptable to diverse patient profiles [4]. The proposed architecture is an initial implementation of a compact, efficient, scalable, and programmable neuromorphic system for preliminary, real-time, automatic, on-chip arrhythmia detection using ECG signals. A comprehensive description of the contributions of this work follow:

- Hardware RTL Model: A hardware model of the proposed neuromorphic hardware system was built using SystemVerilog. The hardware architecture utilizes a delta-modulator based spike encoder and a simple dot-product engine (DPE)-based SNN processor followed by a winner-take-all (WTA) classification module. Key features include online learning via spike-timing dependent plasticity (STDP) and runtime reconfigurability through a scan-chain module, allowing relevant system parameters to be easily modified to suit the classification of other continuous-time

biomedical signals on top of ECG signals. The accuracy of this design was verified using behavioral simulations and timing diagrams from real ECG samples.

- **FPGA Implementation:** The proposed neuromorphic hardware system was implemented on 4 FPGA target boards to record and evaluate the scalability, resource efficiency, timing performance, and power characteristics of the design. Additionally, testing the portability of the design, paired with strong results from FPGA implementation evaluation, promoted the transition to an ASIC implementation.
- **ASIC Implementation:** The hardware design was taken through an automatic, configured RTL-to-GDSII flow to analyze the area, power, and timing results of a chip implementation of this design. This ASIC Implementation aims to prove the efficacy of the proposed design for use in edge devices, primarily through the potential for extremely low power consumption and minimal footprint.
- **Software Model:** A software model was built in Python as a testbench to measure the expected accuracy on ECG classification; additionally, it enabled the opportunity to tune system hyperparameters and identify design improvements to the hardware implementation. The model trains an `snn_torch`-based model of the custom hardware design using recorded, labeled excerpts of ECG data from the MIT-BIH Arrhythmia Database. The trained parameters are passed to a hardware-faithful simulation, which closely mirrors the RTL-level implementation, for testing and evaluating classification accuracy. The software model explores the difficulty of training SNNs, weighted loss functions, methods for weight

quantization, classwise thresholds, and more to provide insight into improving this initial design.

2. BACKGROUND

This section begins by establishing the motivation and potential contributions that this work may have; specifically, it provides a background on cardiovascular diseases and their effect and relevance, as well as the science of Electrocardiograms and the importance of and methods behind its classification. Additionally, this section details the technical background related to these concepts that are relevant for the proposed system – edge devices, different types of neural networks, and concepts in neuromorphic computing. The goal of this section is to establish the motivation and technical background necessary to understand the relevance and theory behind the proposed work and its implementation, detailed in the following sections.

2.1. Clinical Challenge: Cardiovascular Diseases

According to the World Health Organization, Cardiovascular Diseases (CVDs) are the leading cause of death globally [10]. CVDs are a group of disorders of the heart and blood vessels, including coronary heart disease, cerebrovascular disease, peripheral arterial disease, rheumatic heart disease, and many more. Additionally, heart attacks and strokes are acute events that occur due to the blockage of blood flow to the heart or the brain. Usually, this is because of a buildup of fatty deposits on the inner walls of the blood vessels that supply the heart or brain, due to bleeding from a blood vessel, or blood clots.

2.1.1. Global Impact

The most common behavioral risk factors are unhealthy diet, physical inactivity, and tobacco and alcohol use; air pollution is an important environmental risk factor. The effects of these risk factors show up in individuals as increased blood pressure, glucose,

and/or lipids, and obesity, leading to increased risk of heart attack, stroke, or heart failure. Unfortunately, there are no obvious symptoms of CVDs until a heart attack or stroke occur.

More specifically, 80% of the world's deaths from CVDs occur in low/middle income countries [10]. The lack of primary health programs for early detection, treatment for those with risk factors, and effective health care services all contribute to this statistic. As a result, detection of CVD is late, leading to young deaths and a high death toll. This establishes the need for a more accessible method of early detection for these chronic, long-term diseases.

2.2. Electrocardiogram (ECG)

2.2.1. The Electrophysiology of the Heart

Diving deeper into the heart itself, electrophysiology is a branch of cardiology which focuses on the electrical system that regulates heart rhythm [11]. The main purpose of electrophysiology is to identify and treat irregular heartbeats, or arrhythmias, by identifying the faulty electrical circuits in the heart. An electrocardiogram (ECG) signal is a non-invasive, low-voltage recording of the electrical impulses that trigger each heartbeat; it demonstrates the regularity of heart function. Fig. 2.1 below shows the relationship between a cardiac cycle and normal ECG waveform.

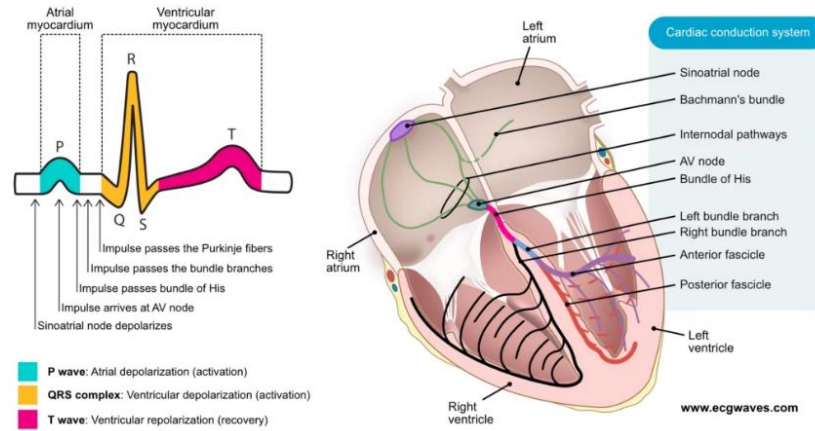


Fig. 2.1: Cardiac Conduction System and ECG Waveform [11]

The cycle begins with the sinoatrial (SA) node, or the pacemaker of the heart [12], discharging an action potential via the atrioventricular node to the ventricles. The impulse spreads through the myocardium and activates the cells which contract. The action potential is what generates the electrical currents which produce the normal or classic ECG waveform; each portion can be traced to the cardiac conduction system. Activation of atria is shown in the P-wave, activation of the ventricles is shown in the QRS complex, and the recovery of the ventricles is shown in the T-wave.

2.2.2. Arrhythmias and Detection

The most important part of ECG interpretation is determining whether the ECG wave and intervals are normal. The morphology (appearance), durations, and intervals of the wave all contribute to the structured assessment of the ECG signal [13]. An irregularity with the rate or rhythm of the heartbeat, which is not physiologically justified, is an arrhythmia. Cardiac arrhythmias can be divided into two categories: bradycardias or tachycardias, which correspond to slower or faster heart rates, respectively [14]. A complex analysis of the various segments, intervals, durations, points, amplitudes and more can be

used to detect various types of arrhythmias. The most relevant measurements gathered from the ECG signal are depicted in Fig. 2.2 below. Detection of an arrhythmia can indicate the presence of a CVD.

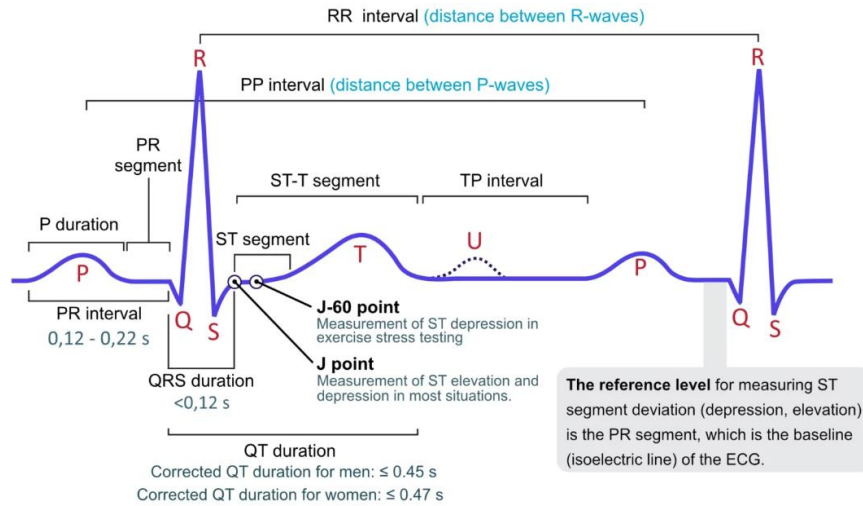


Fig. 2.2: Relevant Measurements from ECG Waveform [13]

Since arrhythmias are often intermittent, traditional diagnosis techniques utilizing the ECG signal have been achieved in different ways. Holter monitors use electrodes to continuously record ECG data for a day or two; experts parse through the recorded data to find any irregularities. Cardiac event recorders are worn for weeks, but only record data when activated. Studies show that 14-day ECG patch monitoring is well tolerated and enables longer continuous monitoring than the 24-hour Holter Monitor, leading to improved clinical accuracy in the detection of paroxysmal arrhythmias [15]. Implantable loop recorders can be used for long-term monitoring to help capture the intermittent arrhythmias. However, these techniques are expensive, inconvenient, and requires analysis from experts. Ultimately, they do not solve the problem of “detection in wild” – detecting arrhythmias that may occur when the clinic isn’t watching.

2.2.3. AAMI ECG Classification and MIT-BIH Dataset

The automatic classification of the ECG signal is essential in performing further analysis and detecting arrhythmias, while overcoming the limitations of manual interpretation. The AAMI EC57 standard, an FDA recognized consensus standard for testing and reporting performance results of cardiac rhythm and ST-segment measurement algorithms [16], establishes a guideline for testing and reporting the performance of cardiac rhythms. It standardizes ECG classification methods to group several individual beat types in 5 major classes: normal beat (N), supraventricular premature or ectopic beat (S), premature ventricular contraction (V), fusion of ventricular and normal beat (F), and unclassifiable or paced beat (Q).

The AAMI classifications are often used on the MIT-BIH Arrhythmia Database [17]. Fig. 2.3 clearly defines ECG signal classification standard specified by ANSI/AAMI EC57, and the number of each class in the dataset [18]. This is a publicly available database containing 48 half-hour, 2 channel ECG recordings, which have been annotated by cardiologists. The data comes from patients at Boston's Beth Israel Hospital from 1975-1979 and was one of the first standard resources for evaluating arrhythmia detectors. The data has been digitized at 360 Hz, with 11 bits of resolution over a 10mV range. The MIT-BIH dataset provides data to test and compare automated ECG classification methods and algorithms.

Groups	ECG classes	Number
N	Normal(N)	75,017
	Left Bundle Branch Block(L)	8,071
	Right Bundle Branch Block(R)	7,255
	Atrial Escape (e)	16
	Nodal (junctional) escape(j)	229
S	Atrial Premature(A)	2,546
	Aberrant Atrial Premature(a)	150
	Nodal (Junctional) Premature(J)	83
	Supra-ventricular Premature(S)	2
V	Premature Ventricular Contraction(V)	7,129
	Ventricular escape(E)	106
F	Fusion of Ventricular and Normal(F)	802
Q	Paced(/)	7,023
	Fusion of Paced and Normal(f)	982
	Unclassifiable(Q)	33

Fig. 2.3: Number of Each ECG Class in MIT-BIH Dataset by AAMI Standard [18]

2.3. The Need for Edge Intelligence

2.3.1. Wearable Cardiac Devices

Wearable technology is developed with the intention of making consumer's lives simpler and promoting beneficial behavioral changes through easy access to many types of important data [19]. The evolution of wearable cardiac devices and the increased popularity and improvement of wearable health monitoring devices have pushed through conventional monitoring to innovative solutions with real-time insights for early detection and proactive healthcare. An investigation on the role of wearables in heart failure (HF) management provides relevant context on findings specific to the monitoring of the ECG signal for arrhythmia detection [20]. VitalPatch evaluates heart performance and sends patient information to a secure cloud for real-time cardiac arrhythmia monitoring [21]. A study found that this means of monitoring enabled the opportunity for earlier health care of atrial fibrillation (AF or AFib), the most common type of treated heart arrhythmia, compared to unmonitored individuals [22]. ZioXT is another patch that records up to two weeks of ECG

data to detect irregular heart rhythms; the data is sent to physicians for analysis. ZioXT was found to be more effective than a 24-hour Holter Monitor in a study for the detection of AF, leading to an increase in clinical accuracy and identification of potential arrhythmias [23]. For monitoring physical activity and exercise, Chest Strap enables precise heart rate measurements and was used to detect arrhythmias in 220 patients and can be used for long-term follow-up to detect AF [24]. The emergence of such devices demonstrates the transition from bulky, wired monitors to wireless patches, showing the possibility to conveniently and continuously read and monitor the ECG signal with high quality. However, the issue still lies in the accumulation of large amounts of data and true, real-time classification without needing expert intervention.

2.3.2. Constraints of Deep Learning at the Edge

Transitioning to automated methods for ECG analysis can be more accurate and quicker than manual examination and can relieve the clinician's workload. However, classic Machine Learning (ML) techniques require hand-crafted features and labels, which can be tedious and expensive. Deep Learning (DL) methods can automatically learn features for classification tasks [25]. Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) have been shown to achieve high accuracy in ECG classification tasks. Some methods use RNNs to leverage the temporal dependency between values in a raw, one-dimensional ECG signal; subsequently, they use CNNs with signal-to-image transformations to explore the classification performance [26]. However, they are computationally expensive and memory intensive. In the context of a battery powered, resource restrained wearable device, the use of power-hungry Artificial Neural Networks (ANNs) often requires offloading data to the cloud; this incurs high energy costs,

raises privacy concerns, and introduces the dependency on a stable network [27]. Spiking neural networks (SNNs) emerge as a plausible neural network paradigm for this problem, with the potential for low-power and local operation.

2.4. Shift from ANNs to SNNs

By comparing the neurons, or main computation units, of Artificial Neural Networks (ANNs) and Spiking Neural Networks (SNNs), the difference between their data representation, computational costs, and temporal dynamics is clear. ANN compute is described by Equation 2.1; the computation between the inputs, x , and weights, w , is a multiply-and-accumulate (MAC) operation [28]. The communication between neurons is achieved by continuous and high precision values; the information is solely propagated as static snapshots in the spatial domain.

$$y = \varphi \left(b + \sum_j x_j w_j \right)$$

Equation 2.1: ANN Compute Process Equation [28]

The spiking neuron has a similar structure to that of the artificial neuron; however, the behavior and communication differ. The data is represented, or communication is achieved, through spike trains coded in binary events as opposed to the continuous activation in ANNs; the sparsity of spikes in SNN-based systems accounts for one of the main advantages of SNNs over ANNs [29]. Due to this difference in data representation, the computation is essentially a gated accumulation. Furthermore, the operation occurs in the spatio-temporal domain – on top of the information propagating in the spatial domain,

the current state, or membrane potential, of the neuron is affected by the history in the temporal domain. The difference between ANNs and SNNs is represented visually in Fig. 2.4.

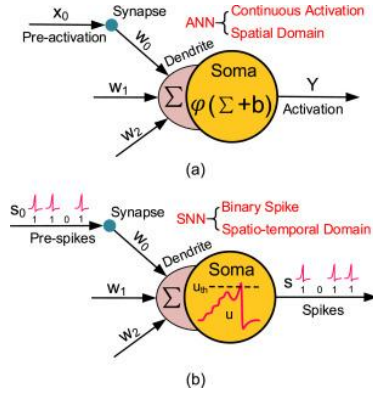


Fig. 2.4: Neuron Model in (a) ANNs and (b) SNNs [28]

2.4.1. Biological Inspiration: Spiking Leaky-Integrate-and-Fire Neuron

Diving deeper into the biological model of the Leaky-Integrate-and-Fire (LIF) Neuron, it strikes a balance between the biophysically accurate Hodgkin-Huxley Neuron Model and the Artificial Neuron Model [28]. As opposed to passing the sum of the weighted inputs into an activation function, like the artificial neuron, the LIF neuron [30] integrates that value over time with leakage. It emits an output voltage spike when the accumulated membrane potential exceeds a threshold. Fig. 2.5 demonstrates the sparse, gated accumulations of pre-neuron weights that occur in the post-neuron, which emits spikes when its membrane potential exceeds the threshold. The occurrence of an action potential is far more important than the variations in the action potential itself [31]. Neurons are connected by synapses, which represents the strength of connection between two neurons. In a biological sense, the neuron requires a sufficient amount of stimulus to become excited enough to fire a spike. Additionally, neurons go through a refractory period

after emitting a spike, where they are unable to accumulate their membrane potential. Information is stored in the timing of the spikes [32], and the LIF neuron provides a balance between biological accuracy and practicality.

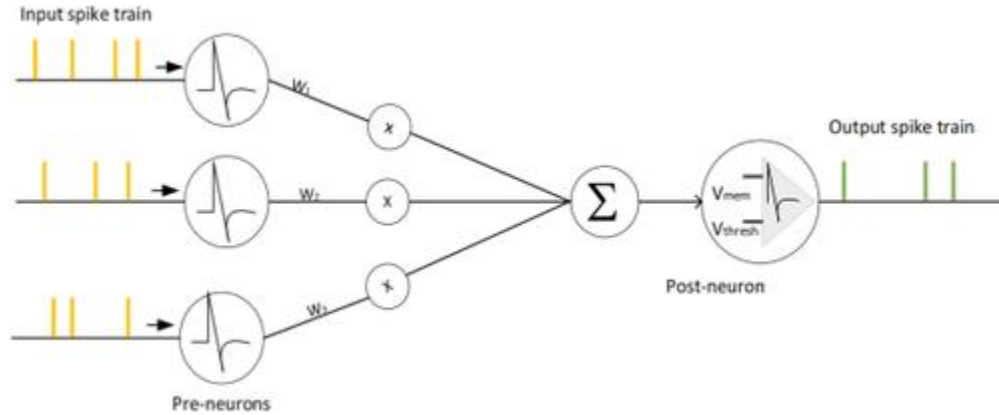


Fig. 2.5: LIF (Post-neuron) Computation in SNN Representation [33]

2.4.2. Advantages of SNNs

Spiking Neural Networks (SNNs) mimic the behavior of biological neurons more closely than traditional neural networks; being closer to natural intelligence, as opposed to artificial models, intuitively promotes the increase in computational efficiency. Additionally, the neuron exists independently and transmits elementary events between each other; therefore, nothing limits the scalability of a SNN system. While ANNs are largely dependent on how they are trained, SNNs can work in a dynamic environment, and are better suited to be adaptable to real conditions, such as processing sensor data. Most importantly, the event driven communication through discrete spikes, lack of huge algebraic computations, and the assumption that information from the outside world is sparse allows the SNN to be a highly power efficient mode of computing that can work extremely fast [34].

2.5. Spike Encoding Schemes

While it is possible to use non spiking inputs for a SNN, there are a few appealing reasons to encode the input into spikes [35]. As we know, biological neurons process and communicate via spikes, which are far simpler to represent and use in hardware than high precision values. Additionally, most neurons spend their time at rest, allowing the silencing of activations due to the sparsity of spikes. Furthermore, sensory periphery only processes information when it changes. In other words, the event-driven processing does not slow down the frequency of sensing, like in conventional signal processing. These attributes of spikes reflect the simplicity in computation, efficiency, and fast processing speeds of neuromorphic hardware with spike inputs.

2.5.1. Rate vs. Temporal Encoding

There is a large amount of research exploring the effectiveness and accuracy of different methods of spike encoding schemes for varying applications [36]; these methods of spike encoding can be generalized to two main modes: rate encoding and temporal encoding [37]. In rate encoding, the information is represented by the average number of spikes in a time window, describing the activity of a neuron. While it is robust to noise and easier to train from ANNs, it consumes a higher amount of energy and has slow inference, making it more suitable for tasks where signal timing is more important than the precision of timing. In temporal encoding, information is embedded in the relative and exact timing of spikes. Unlike rate encoding, temporal encoding exhibits fast inferences and high energy efficiency, at the cost of sensitivity to noise and higher difficulty in training [38]. However,

this justifies its use in neuromorphic hardware and real-time edge applications, where low power consumption is a key requirement.

2.5.2. Delta Modulation

Delta modulation, based on event-driven spiking, can be considered a subset, or time-difference representation, of temporal encoding that effectively converts time series input data into streams of spikes [35]. It generates a binary spike when the difference between consecutive samples exceeds a configurable threshold. If the difference is both positive and greater than the threshold, an ‘on’ or ‘UP’ spike is generated. Conversely, if the difference is negative and less than the threshold, an ‘off’ or ‘DOWN’ spike is generated. Increasing the threshold makes the delta modulator less sensitive to small changes in the input signal. In contrast, decreasing the threshold effectively increases the resolution of the encoding and spike density. The threshold can be tuned to match the input data and effectively extract the necessary features of the sampled input signal. Due to the simplicity of implementation and proven use-cases in similar architectures [39], delta modulation was chosen as the spike encoding scheme for this architecture.

2.6. Theory of the Dot Product Engine (DPE)

The surge of Artificial Intelligence has brought more thought to accelerating the key operations in the computations behind training AI models and AI inference. Vector-matrix multiplication (VMM) dominates the computation time and energy for many of these workloads [40]. The core operation of the dot product is simply a weighted summation of an input vector, x , and a weight vector, w ; additionally, there may be a bias, b . This computation is described in Equation 2.2 below. Traditional ANNs use a high-

precision value for their input, which requires the use of a MAC unit. Additionally, it requires fetching large amounts of weight data to perform each multiplication; often times, the memory bottleneck – the necessary speed for data to move between memory and the processor – can be greater than the compute time. This is known as the Von Neumann bottleneck [41].

$$z = \sum_{i=1}^n w_i x_i + b$$

Equation 2.2: Weighted Summation Equation

2.6.1. Event-Driven Dot Products and Gated Accumulation

The use of a Dot Product Engine in an SNN can help mitigate the effects of both issues – fetching data and expensive computations. The input vector in a DPE based SNN consists of solely 0's and 1's; this reduces the expensive MAC computation into a gated accumulation, where the result of each individual computation is either the locally stored synaptic weight itself, when a spike is present, or 0 when no spike is present. As aforementioned, the sparsity of spikes allows the DPE to remain in a static state for a majority of the time, potentially only consuming leakage power. SNNs are unique in their event-based asynchronous mechanism, with their usually largely parallel and sparsely connected structures [33]. The advantages in terms of energy consumption and computational overhead can be traced to the sparsity of spikes, as well as the inexpensive DPE computation that it utilizes.

To solve the Von Neumann bottleneck that occurs with standard GPU or CPU based vector-matrix multiplication (VMM), the weights in the DPE are stored locally in synapses. By localizing the storage of weights in a distributive memory architecture, these values are more immediately ready for the neuron to accumulate. A localized synaptic array eliminates the memory fetching overhead, in an already event-driven system, establishing the feasibility of a DPE based computation for edge applications.

This DPE computation is often referred to, or thought of, as a crossbar. The resistive crossbar, a simple neuromorphic computing paradigm, is an array of resistive elements which uses Ohm's Law to achieve the multiplication of voltages and conductance, and Kirchoff's Current Law to sum up the currents. The crossbar structure essentially performs the MAC operation, and the principle of the DPE can be extended to a neural network scaling incoming signals to be added, shown in Fig. 2.6 [42]. In an SNN, the spiking postsynaptic neuron accumulates the weights of synapses based on input spikes, acting as a gated accumulation operation as opposed to an explicit MAC operation. The accumulation results in a postsynaptic action potential, or fire, when that value reaches a threshold.

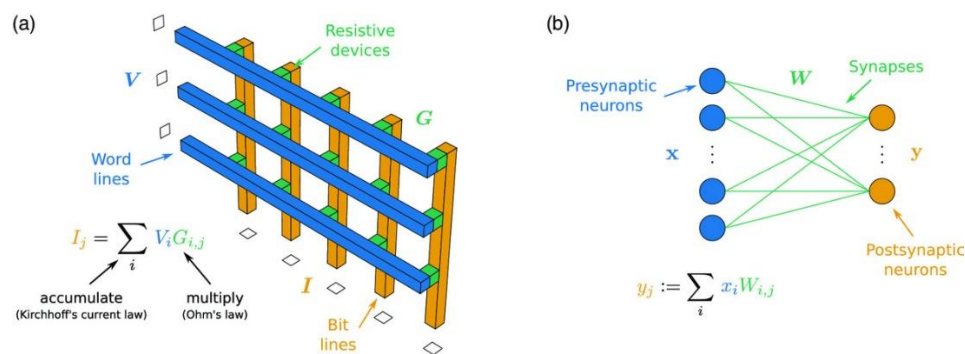


Fig. 2.6: Computing Principles of (a) Crossbar Array based DPE and (b) Fully Connected Synaptic Layers [42]

2.7. Learning Dynamics in SNNs

Learning dynamics in SNNs makes it an adaptive system, as opposed to just a signal processor. These rules aim to govern how the network evolves its state based on the patterns it recognizes. The difference in how information is valued in ANNs and SNNs determines the difference in their learning dynamics. ANNs use training rules such as global backpropagation, where errors are sent back up through the network layers to update neuron weights; this process uses iterative steps, and constant hardware usage. SNNs cannot directly use this approach due to the non-differentiability of spikes. Additionally, simple neuron models used in SNNs struggle to capture rich temporal dynamics. Learnable Neuron Models (LNMs) are a potential solution, allowing integrate-and-fire (IF)-based neuron models to learn its own neuron dynamics during training [43]. Furthermore, learning principles [44] can utilize the fact that the synaptic weight between two neurons represents the strength of their connection to only update weights based on the neurons they are in contact with; this is the concept of local plasticity. This allows the network to stay dormant until a significant temporal event, allows for locally transparent learning, and exhibits temporal sensitivity, as time is built into the way the neurons work.

2.7.1. Spike Timing Dependent Plasticity (STDP)

The biologically inspired Spike-Timing Dependent Plasticity (STDP), a Hebbian learning rule [44], serves as a strong alternate to bypass the non-differentiable nature of spikes. STDP is an unsupervised learning rule that updates the synaptic weight based on the timing of spikes between the pre-synaptic and post-synaptic neurons [45]. A linear approximation of this rule can be used for greater efficiency. When a pre-synaptic spike

precedes a post-synaptic spike, with a certain learning window, Long Term Potentiation (LTP) occurs. This strengthens that synaptic weight, since the pre-synaptic spike closely and directly contributed to the firing of the post-synaptic spike. Conversely, when the post-synaptic spike occurs closely before the pre-synaptic spike, Long Term Depression (LTD) happens. This is the weakening of the synaptic weight, since the pre-synaptic spike did not directly contribute to the firing of that post-synaptic spike. The magnitude of synaptic weight change is inversely proportional to the timing difference between spikes; a shorter timing difference would induce a greater amount of strengthening or weakening of synaptic weight [46]. A non-linear visualization of STDP learning is shown in Fig. 2.7, where an exponentially increasing amount of weight change occurs when the difference in timing between the presynaptic and postsynaptic spikes is shorter [47].

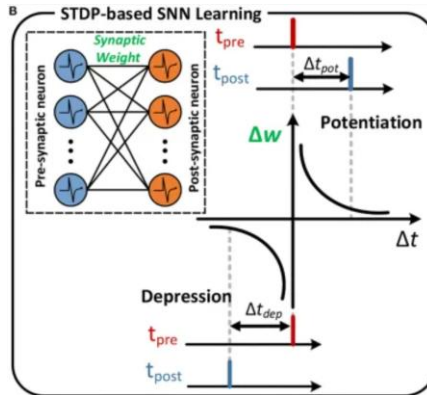


Fig. 2.7: STDP based Learning in SNNs [47]

2.7.2. Competitive Learning – Winner-Take-All (WTA)

Competitive learning through a Winner-Take-All (WTA) Circuit can extract a final decision from several neurons to choose the strongest feature. It stems from lateral inhibition [48] - the biological process where active neurons suppress the activity of their

neighbors to enhance signal contrast and improve perceptual clarity and accuracy. The WTA uses this principle to create competition between a group of neurons, where the neuron with the highest activation potential, the first neuron to fire a spike, or the neuron that fires the most within a window, wins and transmits its signal, while the activity of the other neurons is suppressed. This stimulates the lateral inhibition and competition mechanisms of biological neural systems to provide an effective method for information encoding and processing in SNNs.

2.8. Related Works

Several research papers and studies contributed to the decisions that shaped design choices and hardware architecture; however, a few key pieces of literature heavily considered all portions of this system – ECG classification for wearable, edge devices utilizing neuromorphic principles and SNNs. A concise summary of these pieces of literature follows.

Galvis-Chacón et al. [1]. proposes the use of SNNs with axonal delays (D-SNNs) for ECG signal classification by using the LIF neuron with axonal delays on the MIT-BIH Arrhythmia Database. This approach features improved energy efficiency, asynchronous event-driven processing, and a biologically inspired approach to signal classification. The work by Gerber et al. [4] expands the SNN-based ECG anomaly detection task by extending the memory capacity of the SNN using parallel delay chains to show how temporal information is properly preserved in the network activity. It addresses the intrinsic memory challenges of processing slow, multi-second physiological signals on neuromorphic hardware.

Banerjee et al. [3] explores the design and implementation of a Reservoir based SNN and a Feed-forward SNN based ECG classifier for wearable edge devices. ECG pattern classification performance was compared with the different SNN architectures, as well as different spike encoders (peak-based and two others). This work provides a comparative foundation for the spike-encoding frontend of the proposed architecture. Chu et al. [5] proposes a SNN processor for always-on ECG classification, using level crossing (LC) sampling to capture the ECG signal, spatio-temporal back propagation (STBP) as the training scheme, and a hierarchical memory access scheme. This work provides the blueprint for always-on, digital backend VLSI architectures; the hardware architecture proves that co-optimizing network sparsity with customized memory layout can achieve an extremely low-energy threshold per classification for edge processing.

3. HARDWARE ARCHITECTURE & DESIGN IMPLEMENTATION

This section provides an overview of the proposed hardware system in its final, optimized implementation. For every module that makes up this system, a description of the way it works, and its implementation is effectively defined, with the aid of code blocks and timing diagrams. The goal of this section is to provide a high-level understanding of the way the entire system operates, and a unit level understanding of the inner workings and neuromorphic computing principles implemented in this design.

3.1. System Overview and Implementation

The proposed design is a compact, scalable, and programmable neuromorphic hardware system for preliminary real-time arrhythmia detection from ECG signals; a block diagram of the proposed system is shown in Fig. 3.1. The architecture uses a delta-modulator-based spike encoder to convert discretized ECG signals into two, input spike streams. These spike streams pass through the dot-product engine (DPE) based spiking neural network (SNN) processor, consisting of synapse units, accumulator units, and compare and fire units; these three modules act as the synapses – localized memory with online spike-timing-dependent plasticity (STDP) learning – and leaky-integrate-and-fire (LIF) neurons, which accumulate the synaptic stimuli and emit an output spike when that value exceeds a threshold. This is followed by a winner-take-all (WTA) module, which takes in the 5 output spike streams from the LIF neurons and provides the final classification, or decides on one column as the winner. Additionally, all modules are encompassed by a scan chain wrapper (SCW) to provide runtime reconfigurability for our system and allow key system parameters to be tweaked; this provides the basis for applying

this system to other continuous-time biomedical signals other than ECG signals. The proposed architecture demonstrates the potential to resolve some of the shortcomings of traditional and deep learning classification models, while dealing with the complexities of real-time signal encoding and hardware-level reconfigurability required for diverse patient profiles. The hardware design implementation discussed in this section is the optimized version; an overview of the iterations to reach this architecture is discussed in Section 4.

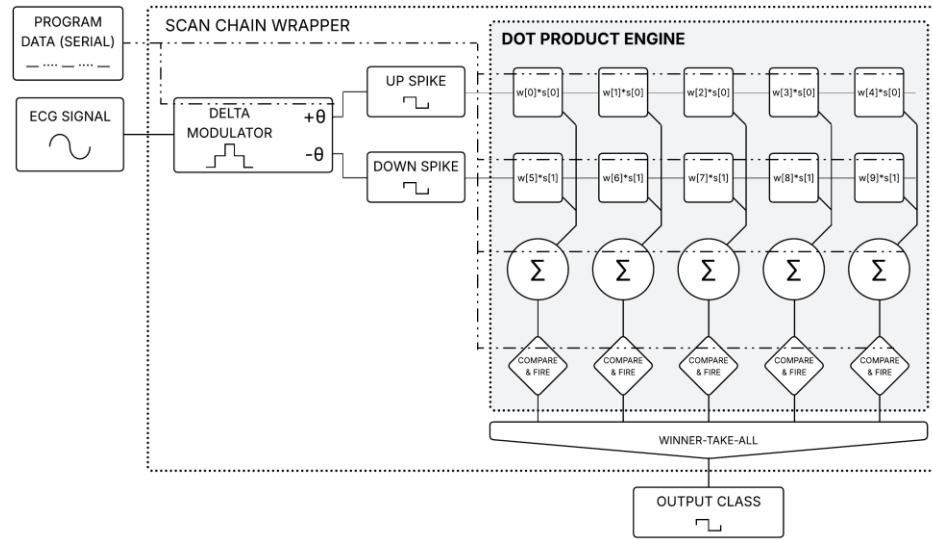


Fig. 3.1: Hardware Architecture Block Diagram

3.1.1. Processing Elements

This subsection describes the submodules, or the processing elements, that make up this system; this includes a delta modulator as the spike encoding scheme, synapse unit and accumulator as a part of the DPE, and WTA classification module.

3.1.1.1. Delta Modulator

The delta modulator effectively converts input, discretized, 11-bit ECG data into two output binary spike streams; the maximum decimal value is 2047 and the minimum

decimal value is 0. It identifies significant temporal changes in the ECG signal and generates spikes when the deviation compared to the locally tracked signal reconstruction, 'signal', is large enough. The 'signal' register acts as a local estimator; it is initialized with the first ECG signal value, 'ecg_in', and is compared to the next value every subsequent cycle. The spiking behavior is governed by the size of the threshold, 'STEP_SIZE', which determines the sensitivity of encoding; the decimal value chosen for the 'STEP_SIZE' is 4. When the difference between the current sample and tracked signal reconstruction value exceeds the threshold, 'signal' increases or decreases by 'STEP_SIZE', depending on if the sample is greater than or less than the signal and threshold, respectively. The combinational next value logic and sequential conditional statements corresponding to this comparison is shown in Code Block A below. Additionally, a dead cycle, or zero value, was inserted between spikes to help the downstream dynamic learning and stability. The result is 2 single bit spike streams, 'UP' and 'DOWN', corresponding to increases and decreases in the sampled ECG value.

```

// NEXT 'UP' SPIKE SIGNAL VALUE
wire [DATA_WIDTH-1:0] next_up = (signal + STEP_SIZE > MAX_VAL) ?
    MAX_VAL : (signal + STEP_SIZE);

// 'UP' (dm_spike_out[0]) SPIKE CONDITIONAL STATEMENT - SEQUENTIAL
if (ecg_in > signal + STEP_SIZE) begin
    dm_spike_out[0] <= 1'b1;
    last_cycle_spiked <= 1'b1;
    signal <= next_up;
end

// NEXT 'DOWN' SPIKE SIGNAL VALUE
wire [DATA_WIDTH-1:0] next_down = (signal < STEP_SIZE) ?
    MIN_VAL : (signal - STEP_SIZE);

// 'DOWN' (dm_spike_out[1]) SPIKE CONDITIONAL STATEMENT - SEQUENTIAL
if (ecg_in + STEP_SIZE < signal) begin
    dm_spike_out[1] <= 1'b1;
    last_cycle_spiked <= 1'b1;
    signal <= next_down;
end

```

Code Block A

The operation of the delta modulator is shown in Fig. 3.2. After initialization, the testbench processes an ECG sample from the MIT-BIH Arrhythmia Database at every positive edge of the clock. It is reconstructed in the delta modulator as ‘signal’ and is evaluated against the configurable threshold ‘STEP_SIZE’. The ‘next_up’ and ‘next_down’ wires precompute the potential next value for signal, for if the difference between the next sample and reconstructed signal is greater than or less than the threshold, respectively. This condition triggers either an ‘UP’ spike, shown by ‘dm_spike_out[0]’, or a ‘DOWN’ spike, shown by ‘dm_spike_out[1]’. Additionally, the ‘last_cycle_spike’ signal tracks these spikes to force the spike streams to be pulsed, for better processing in the rest

of the system. The simulation reflects that the sampled ECG data can effectively be tracked and represented in the two spike streams by the presence and frequency of spikes.

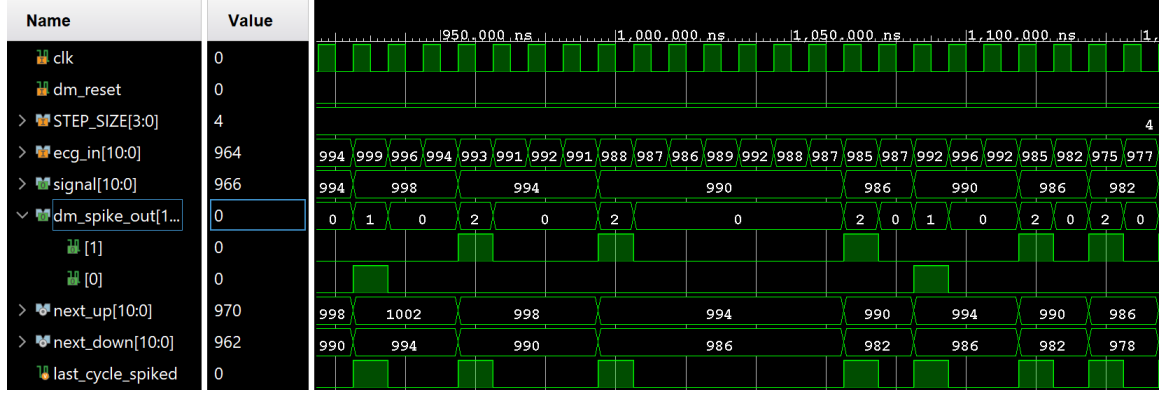


Fig. 3.2: Delta Modulator DUT Simulation Waveform

3.1.1.2. Dot Product Engine based Spiking Neural Network

The spiking neural network (SNN) is implemented using a dot product engine (DPE), which is accomplished using a neuron array along with a synaptic crossbar. The architecture for this implementation is a 2x5 synaptic array. The two rows correspond to the differential spike streams from the delta modulator, while the five columns represent the output classes for automatic ECG classification. The array operates synchronously; each column includes connections to input spikes and output neurons using synapses that allow online learning computed with a ‘neuron_time’ signal, tracking the time since the last postsynaptic action potential. This existing design is parameterized and can easily be modified to implement different numbers of inputs and outputs in a DPE configuration for different biomedical signal classification applications. In addition to that, multiple DPEs can be cascaded to create a deep spiking neural network. The main building blocks of this DPE are the synapse units and the neuron units – each consisting of an accumulator and a compare-and-fire module – which are described in detail below.

3.1.1.2.1. Synapse Unit

The synapse units act as the localized memory of the DPE and serve two main functions: inference gating and plasticity logic. The gating logic determines how only the presence of an input binary spike from the delta modulator triggers a 4-bit, signed synaptic weight contribution to the accumulator. This synaptic weight represents the strength of connection between two neurons; in this case, it is the strength of contribution from the delta modulated input spike stream to a certain output classification. Aside from general module inputs (clock, enable/config signals, etc.), the synapse unit module takes the input spike, time since the last post synaptic action potential ('neuron_time'), and initial weight (from the scan chain wrapper) as inputs.

The time since the last input spike or presynaptic action potential, 'synapse_time', is tracked with a counter to contribute to the STDP computation and update synaptic weights. The weights are updated based on the timing difference between the 'neuron_time' and 'synapse_time' using a look-up-table. The resulting value from the look-up-table is greater in magnitude if the timing difference between the presynaptic and postsynaptic spikes is shorter. This implementation of STDP serves as an efficient, low-cost linear approximation of STDP. Additionally, the synaptic weights are only updated if certain criteria are met every clock cycle; the STDP calculation and respective update only occurs when STDP is enabled, within a configurable learning window before or after a postsynaptic action potential, and if either an input spike is detected within a specific range of a postsynaptic action or an output spike is detected. The STDP look-up-table, timing difference computation, and logic for valid STDP calculations are shown in Code Block B. Every clock cycle, the synapse updates its own 'synapse_time' counter to reset if an input

spike is detected or increment if not. If the `stdp_calc` signal is valid, the synaptic weight is updated by adding the appropriate value from the look-up-table to the previous weight. The ‘`stdp_calc`’ and ‘`lut_index`’ signals are computed combinatorially but registered every clock cycle to be used for updating weights to shorten the logic path. Finally, the weight is passed to the downstream accumulator module if a spike is detected, since no weight change would occur if there is no detected spike. The inference gating based on the binary nature and sparsity of spikes enables the saving of dynamic power, as system updates and computations only occur when necessary.

```
// STDP LUT
typedef logic signed [3:0] lut_t;
localparam lut_t stdp_lut [0:6] = '{1,2,3,0,-2,-1,0};

// STDP LOGIC FOR  $\Delta t$  - COMBINATIONAL
delta_t = $signed({1'b0, neuron_time}) - $signed({1'b0, syn_time});
// STDP LOGIC FOR VALID STDP CALCULATION AND LUT INDEX - COMBINATIONAL
if ((delta_t < -LEARN) || (delta_t > LEARN)) begin
    lut_index = LEARN;
    stdp_calc = 1'b0;
end else begin
    lut_index = delta_t + LEARN;
    stdp_calc = stdp_enable &&
        (((spike_in != 0) && (neuron_time < LEARN))
        || (neuron_time == 0));
end

// WEIGHT TO BE PASSED TO ACCUMULATOR - COMBINATIONAL
assign weight_to_accum = (enable && spike_in != 0) ?
    ($signed(weight_reg)) : '0;
```

Code Block B

The waveform depicted in Fig. 3.3 shows the internal signals of one synapse unit instantiation; specifically, the 6th, or second row, second column synapse unit. The blue

markers show the two instances in this period where a postsynaptic action occur, depicted by the ‘neuron_time’ reset to 0. The ‘syn_time’ signal counts up from 0 after every presynaptic spike, shown in ‘spike_in’. The ‘delta_t’ signal computes the difference between the postsynaptic neuron spikes and presynaptic input spikes. The ‘stdp_calc’ and ‘lut_index’ signals determine when an STDP update should occur and the appropriate LUT index for the update, and the registers for those signals update the synaptic weight, seen in the ‘weight_reg’. The working STDP logic can be intuitively seen as follows: the proximity of an input spike to the first fire initially increases the weight from 8 to 11. Shortly after, another spike triggers the decrease of the weight from 11 to 9. The second fire detects an input spike within the range for STDP learning and increases the weight from 9 to 10. Evidently, the timing of spikes affects the amount of weight increase or decrease that occurs. Additionally, the gating logic can be seen in the ‘weight_to_accum’ signal; it only passes the synaptic weight when there is an active input spike; otherwise, the value being passed to the accumulator is 0.

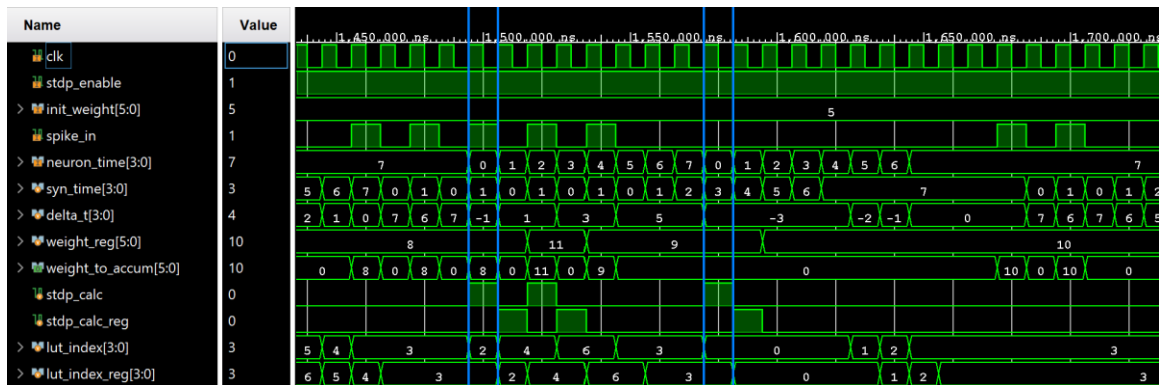


Fig. 3.3: Synapse Unit DUT Simulation Waveform

3.1.1.2.2. Accumulator / Compare and Fire

The combination of the accumulator unit and compare and fire unit acts as the postsynaptic neuron, or LIF neuron. The neuron receives gated, synaptic weight contributions from the synapses in its column and accumulates these values. The accumulated value, or membrane potential, experiences leaking, where the membrane potential periodically moves towards 0. When the membrane potential exceeds a configurable threshold value, the neuron emits a postsynaptic action potential, or fires an output spike. When an output spike occurs, the neuron goes through a refractory period, where it inhibits its ability to accumulate synaptic stimuli for a period of time directly after the output fire. The hardware implementation of the LIF neuron follows.

The ‘weight_to_accum’, or gated synaptic weights, in each column are passed to the accumulator; each accumulator has two synapses contributing to its membrane potential. This input is parameterized to handle the scaling of array size. Additionally, the necessary system parameters for the LIF neuron (refractory period, leaking rate, threshold) are input to the module as well. Like the way the synapse unit only updates the synaptic weight under certain conditions, the accumulator only integrates values when the neuron is not under a refractory period, using the ‘accum_on’ logic; mimicking how biological neurons enter a temporary, unresponsive state after firing an action potential, the accumulator inhibits its ability to accumulate values, or synapses do not contribute to accumulation, for a configurable amount of time after firing. Every clock cycle, the two ‘weight_to_accum’ signals from the synapse units are added and stored in ‘pre_accum’. The ‘next_accum’ value accumulates the new synaptic weight contribution to the previous accumulated value and is assigned with a non-blocking statement to the ‘accum_reg’ when

the value does not need to leak. To incorporate leaking to the membrane potential, the accumulator takes an input, 'leak_rate'. Every 'leak_rate' number of clock cycles, the membrane potential moves towards 0 by 1 decimal value. This means if the accumulated value is positive, it decreases by 1, and if the value is negative, it increases by 1. If the accumulated value happens to be at 0, no leaking occurs. The leaking operation can be modified to leak towards zero more rapidly or slowly, depending on the frequency of new synaptic weights to be integrated. This suggests that a certain temporal density is required to drive the membrane potential to the firing threshold. The implementation of the key pieces of sequential logic is shown in Code Block C below.

The compare and fire module/operation has been implemented within the accumulator module. Every clock cycle, the stored accumulator register value is evaluated against the firing threshold to determine whether the output neuron should fire a spike, or its action potential. The output spike emitted lasts for one clock cycle, and it triggers the reset of a neuron's own membrane potential as well as the leak counter. Additionally, the accumulator takes an input from the WTA circuit, 'any_fire', to signify the firing of a different neuron to inhibit the membrane potential of the other neurons. Since the output spike triggers the reset of the 'neuron_time' counter, the refractory period is applied and the 'accum_on' signal is low until after the period is over.

```

// ADD SYNAPSE VALUES - SEQUENTIAL
if (reset)
    pre_accum <= '0;
else begin
    automatic logic signed [b_accum+1:0] sum = '0;
    for (int i = 0; i < N_inputs; i++)
        sum = sum + $signed(syn_in[i]);
    pre_accum <= sum;
end

// LOGIC FOR VALID STATE TO UPDATE ACCUMULATED VALUE - COMBINATIONAL
assign accum_on = enable && (neuron_time > refractory - 1'b1);

// IF ACCUM_ON IS VALID, CALCULATE NEXT VALUE - SEQUENTIAL
automatic logic signed [b_accum+1:0] next_accum = accum_reg + pre_accum;
// ASSIGN NEXT VALUE IN LEAK CLOCK CYCLE - SEQUENTIAL
accum_reg <= (next_accum > 0) ? (next_accum - 1'b1) :
    (next_accum < 0) ? (next_accum + 1'b1) : next_accum;

// COMPARE AND FIRE IMPLEMENTATION
// FIRE WHEN ACCUMULATOR EXCEEDS THRESHOLD/KILL - SEQUENTIAL
if (fire)
    fire <= 1'b0;
else
    fire <= enable && (accum_reg >= $signed(threshold));

```

Code Block C

The waveform in Fig. 3.4 shows the signals in an accumulator instantiation, which performs the gated accumulation of two synapses, with leaking and a refractory period; additionally, it fires an output action potential if the membrane potential exceeds the threshold. In this example, the refractory period is set at 2 clock cycles after firing, the leak rate is set to 4 cycles after a leak, and the threshold is set to 20. The two synapse inputs to the accumulator are seen in 'syn_in[0:1]', and processed to 'pre_accum'. The 'accum_reg'

sequentially accumulates the synaptic stimuli, and ‘accum_out’ is combinationaly assigned to ‘accum_reg’. The ‘any_fire’ signal signifies that another column fired an action potential, which triggers the reset of the membrane potential. Once the accumulated value in ‘accum_reg’ exceeds the threshold of 20, the neuron fires its action potential on the next positive edge of the clock, and the membrane potential is reset on the following clock edge. This triggers ‘accum_on’ to switch off, and the neuron to enter its refractory period, where it is unable to accumulate any synaptic input for 2 clock cycles. Additionally, the leaking of the neuron membrane potential can be seen every fifth clock cycle; an example of this, seen in the ‘accum_reg’, is the drop from 10 to 9, and the increase from 19 to 23 instead of 24. Computed based on the ‘fire’, the neuron_time resets to 0 the clock cycle after a ‘fire’.

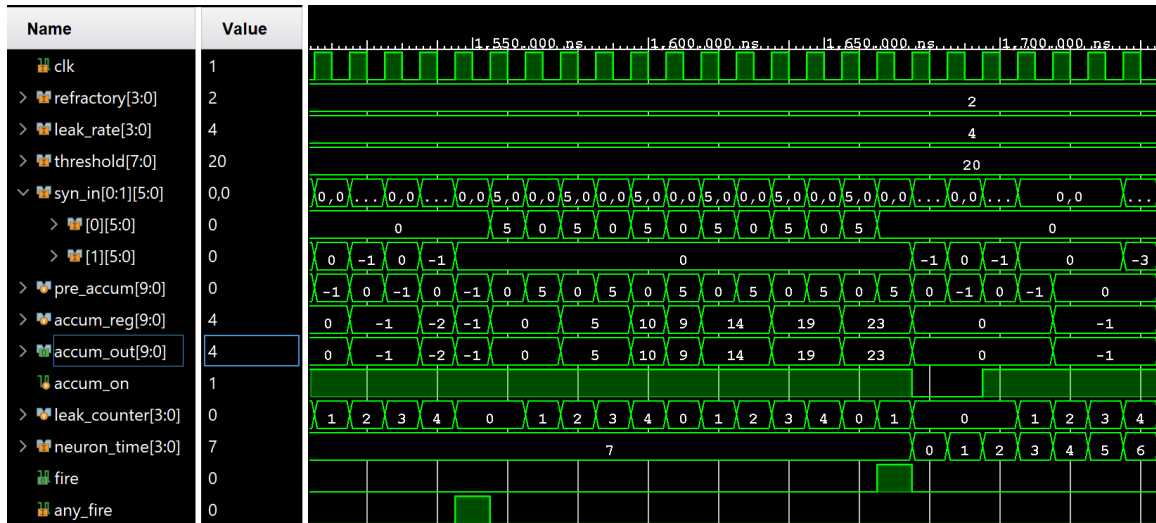


Fig. 3.4: Accumulator DUT Simulation Waveform

3.1.1.3. Winner-Take-All

The winner-take-all (WTA) module determines the single output classification based on first spike priority encoding using the 5 columns of output spike streams generated by the accumulator module. This module is strictly combinational and simply

loops through the 5 column output spike streams to find a spike. It outputs the ‘winner’, as well as the reset signal to the accumulator modules to inhibit the other columns.

The simulation in Fig. 3.5 shows the WTA circuit choosing a class based on the 5 ‘fire’ action potentials produced by the accumulator. When multiple action potentials occur at once, the WTA chooses based on the priority encoding; it chooses fire[2] over fire[4] in the second firing case. The WTA effectively and combinationally chooses an output classification.

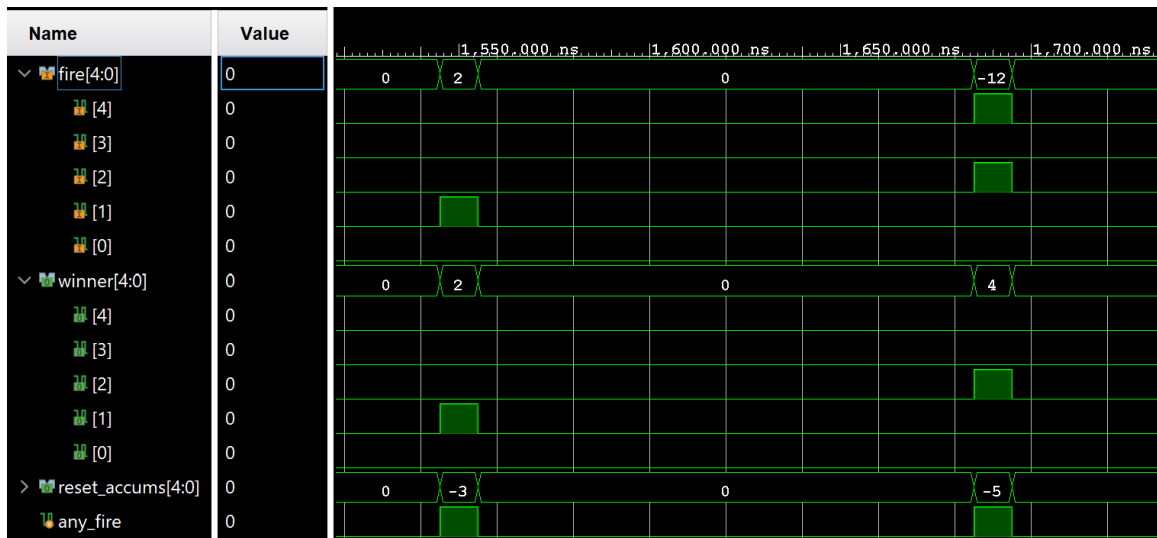


Fig. 3.5: WTA DUT Simulation Waveform

3.1.2. Top-Level Modules

This subsection describes the top-level modules that instantiate and connect the processing elements to create this system; the neuron array builds the DPE based SNN and WTA, while the scan chain wrapper integrates the neuron array with the delta modulator and scan-chain interface.

3.1.2.1. Neuron Array

The neuron array module serves to instantiate the entire DPE based SNN and WTA. This module takes in the following system parameters from the Scan Chain Wrapper (SCW): initial synapse weights, threshold, refractory period, leak rate, and a 'load_config' signal. Additionally, it requires the two delta modulated input spike streams. It outputs the 5 columns of 'fire' (for debugging) and 'winner'. This higher-level module owns the logic for tracking the postsynaptic neuron time, which increments by 1 every clock cycle to a maximum decimal value of 7. It uses the 'fire' signal from the column accumulators to reset when a postsynaptic action potential is detected. Next, it has the enable logic for the 3 modules it instantiates (synapse units, accumulators, and the WTA) to turn on only after the scan chain wrapper is finished safely loading the system parameters, using the 'load_config' signal. Next, it generates 10 instantiations of synapse units and 5 instantiations of accumulators, which are connected by the 'syn_out' signal, shown in Code Block D; this signal connects the 'weight_to_accum' output from the synapse units to the 'syn_in' input to the accumulators. Finally, it instantiates the WTA module. The size of the neuron array (and downstream instantiations) is parameterized, allowing the system to be configured to different sizes.

```

// LOGIC SINGAL CONNECTING SYNAPSE OUTPUT TO ACCUMULATOR INPUT
logic signed [b_weight+1:0] syn_out [N_inputs*N_neurons];
// INSIDE SYNAPSE UNIT DUT - OUTPUT WEIGHT TO ACCUMULATE
.init_weight(synapse_weights[N_neurons*i+j]),
.weight_to_accum(syn_out[N_neurons*i+j])

// SYNAPSE UNIT TO ACCUMULATOR PARAMETERIZED CONNECTION
logic signed [b_weight+1:0] syn_col [N_inputs];
for (genvar r = 0; r < N_inputs; r++) begin : gen_syn_map
    assign syn_col[r] = syn_out[N_neurons*r + k];
end
// INSIDE ACCUMULATOR DUT - SYNAPSE STIMULI TO ACCUMULATE
.syn_in(syn_col),

```

Code Block D

The simulation in Fig. 3.6 shows the delta modulated input spikes streams, ‘spike_in[1:0]’, synaptic stimuli from the 10 synapses, ‘syn_out[0:9]’, accumulated membrane potentials in the 5 column neurons, ‘accum_cols[0:4]’, 5 columns of neuron postsynaptic action potentials, ‘fire[4:0]’, and WTA classification, ‘winner[4:0]’. Each column of 2 synapse weights, membrane potential, and output fire signals are color coded to better visualize the data flow through the system. Synapses 0 through 4 see the ‘UP’ input spike stream from ‘spike_in[0]’ and synapses 5 through 9 see the ‘DOWN’ input spike stream from ‘spike_in[1]’. The first postsynaptic action shown in magenta, ‘fire[4]’, occurs directly due to the presynaptic spikes in ‘syn_out[9]’, which triggers long-term potentiation (LTP) and increases the weight from 3 to 6. Conversely, ‘syn_out[4]’ spikes shortly after ‘fire[4]’, triggering long-term depression (LTD) from -2 to -4 since it had no contribution to the neuron firing. This firing event triggers the reset of all accumulators, and ‘accum_cols[4]’ enters a refractory period; it ignores the first synaptic stimuli from ‘syn_out[4]’ due to this. Each membrane potential can be seen accumulating the synaptic

stimuli that it receives, while applying a period leak. A similar situation occurs for the second firing event pipeline, ‘fire[1]’ shown in red. The synaptic stimuli from ‘syn_out[1]’ directly contributes to the ‘fire[1]’ action potential, so LTP occurs to strengthen the weight from 5 to 8. LTD decreases the weight of ‘syn_out[6]’ from 1 to -1, since it spikes shortly after the firing event. Again, all columns are inhibited, and ‘accum_cols[1]’ enters the refractory period. Seen at the bottom, the WTA classification, ‘winner[4:0]’ mimics fire in this scenario with distinct, single firing events. This simulation waveform demonstrates the entire signal processing pipeline that occurs in the DPE based SNN array to WTA output from delta modulated input spike streams.

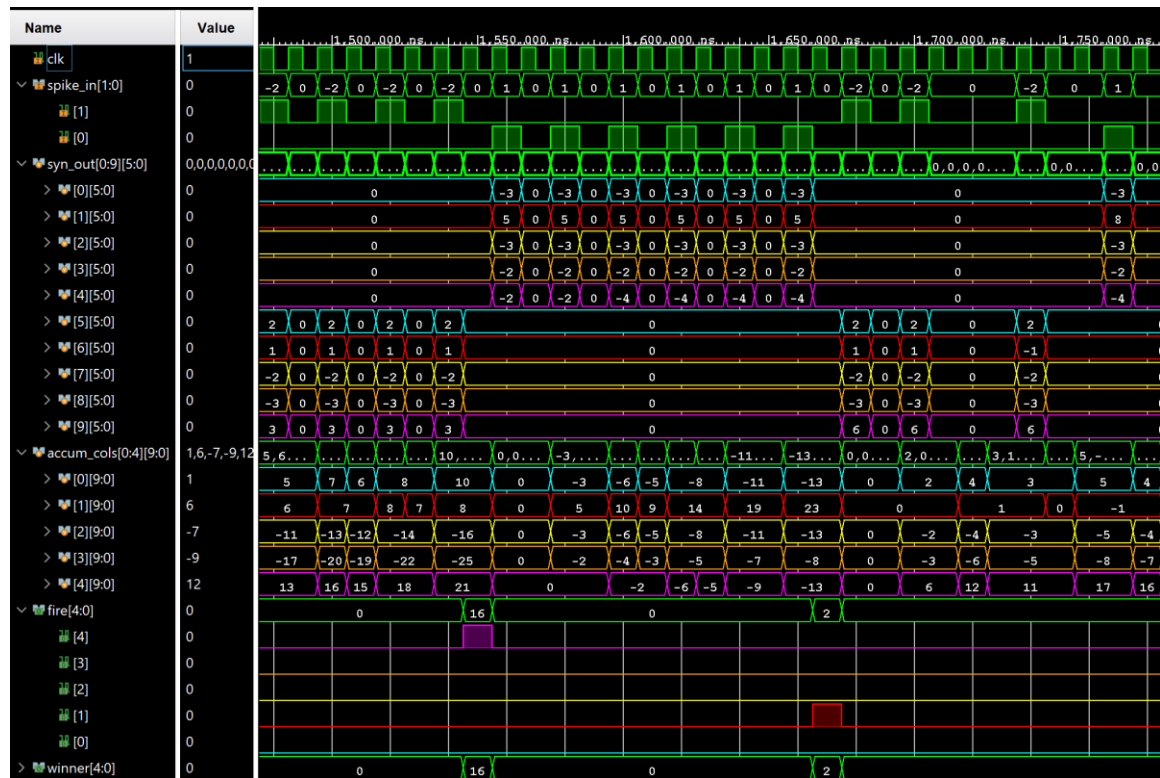


Fig. 3.6: Neuron Array DUT Simulation Waveform

3.1.2.2. Scan Chain Wrapper

The scan-chain wrapper (SCW) is the topmost module, taking in the clock, reset signals, SCW signals (clock, data, and load configure signals), and the ECG input sample; it outputs the 5 columns of 'fire' and 'winner' – only the 'winner' output is necessary. The architecture of the SCW uses two registers to serially read the program data, and parallelly load or assign it to the desired system parameters. Upon reset, the 'shift_reg' reads the binary value passed in from the 'prog_data' on every positive edge of the 'prog_clk'. Once the data has been passed into the 'shift_reg', the 'load_config' signal triggers its capture to the 'config_reg'. The parameter data was scanned in from MSB to LSB, starting with the MSB of the highest index weight to the LSB of the lowest index weight, followed by the threshold, refractory period, leak rate, and delta modulator threshold. The appropriate data is assigned to each system parameter logic signal, which is passed in with the instantiation of the delta modulator and neuron array modules. The sequential loading of data to the 'shift_reg', assignment to the 'config_reg', and bit packing protocol to system parameter logic signals is shown in Code Block E.


```

// READING EACH BIT OF PROGRAM DATA SERIALY - SEQUENTIAL
shift_reg <= {shift_reg[TOTAL_BITS-2:0], prog_data};

// LOADING CONFIG_REG
if (load_config)
    config_reg <= shift_reg;

// ASSIGNING PROGRAM DATA TO SYSTEM PARAMETER SIGNALS
offset = TOTAL_BITS-1;

synapse_weights = config_reg[offset -: (N*B_WEIGHT)];
offset -= (N*B_WEIGHT);
threshold = config_reg[offset -: B_THRESH];
offset -= B_THRESH;
refractory = config_reg[offset -: B_REFRAC];
offset -= B_REFRAC;

leak_rate = config_reg[offset -: B_LEAK];
offset -= B_LEAK;

```

Code Block E

The simulation waveform in Fig. 3.7 shows the processing of data to the scan chain wrapper to be passed to the downstream parameters. Once the system is reset, `prog_data` is passed, bit-by-bit, to the internal ‘`shift_reg`’ using the ‘`prog_clk`’. The ‘`load_config`’ signal signifies that the data is finished being processed, and the current state of the ‘`shift_reg`’ is captured by the ‘`config_reg`’. The bits in the ‘`config_reg`’ are combinationally assigned to the system parameter logic signals. As shown in the value column after ‘`load_config`’ goes high, the synapse weights, threshold, refractory period, leak rate, and delta modulator threshold all receive their correct values. Once this process is completed, the delta modulator is reset and able to begin processing the ECG signal input.

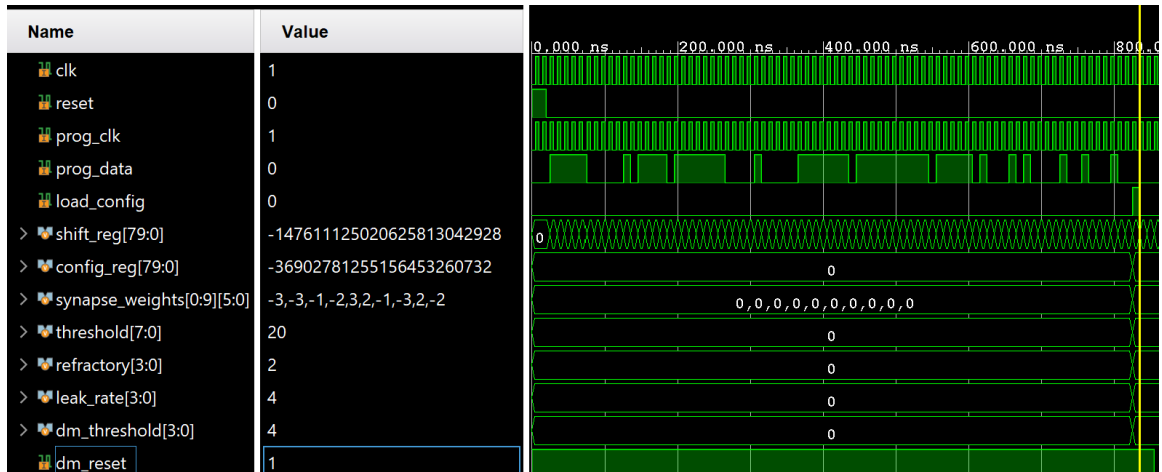


Fig. 3.7: Scan Chain Data Processing Simulation Waveform

The simulation waveform in Fig. 3.8 shows the continuation of the SCW DUT after the system has been initialized. The ECG input, ‘ecg_in’, is processed by the delta modulator into the two spike streams, ‘dm_out[1:0]’. The spikes are processed through the neuron array described in Section 3.1.2.1, and the output ‘fire’ action potentials and ‘winner’ WTA classification signals are output by this top module. This demonstrates the working state of the entire system, from initialization via scan chain to output spike classification.

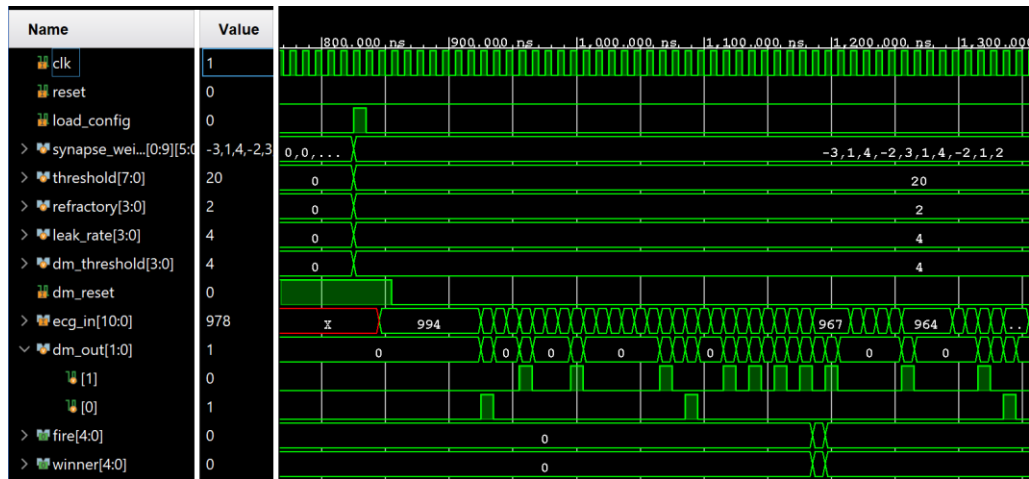


Fig. 3.8: Scan Chain Wrapper DUT Simulation Waveform

4. HARDWARE OPTIMIZATIONS

This section details the process that resulted in the optimized design established in Section 3. It cycles between describing and analyzing the current design with timing diagrams, and describing the goals for the following iteration in terms of design accuracy and performance, as well as the reasoning behind the proposed changes, until reaching the final version of the architecture. The goal is to give an insight into the process that led to the final version of the design, by revealing the considerations made at each step.

4.1. Design Constraints vs. Optimization Goals

As in any design, there was a distinction between the hard design constraints that were established versus the optimization goals, or the metrics that were maximized. The most important constraint was the design accuracy. Once we established how the information (spikes) should propagate throughout our system, any further optimizations ensured that the design was still accurate. This accuracy refers to avoiding the presence of any incorrect spikes, unnecessary delays or unwanted latency, missed spikes, incorrect calculations (STDP, accumulation, leaking), and more. The result of establishing the accuracy baseline was a minimum acceptable performance level that could be evaluated through simulation and timing diagrams through every optimization and design iteration. This process was a cycle between design, simulation, and evaluation to determine where the design was falling short and needed improvement. From there, optimization goals to improve the timing (maximum frequency), lower the resource utilization and power consumption were introduced by modifying the code logic.

4.2. Hardware Logic Tradeoffs

One of the main distinctions that allowed for the improvement of this design was establishing in what situations and where latency was acceptable versus where it was unnecessary. This distinction allowed the breaking up of long, combinational paths of logic – mostly related to the STDP weight update and membrane potential integration – by inserting registers at critical junctions. While combinational logic can finish a task in “zero” clock cycles, it can also create a long Worst Negative Slack (WNS) path. The effect of resolving this issue was improving the maximum frequency at the cost of some signal latency due to the addition of pipelining stages. However, as long as the timing logic and registered data accounted for the use of a registered update strategy, where the decision to make an update and the actual update occur in consecutive cycles, there can be no missed data and no negative effect on the design accuracy.

4.3. Architectural Evolution

This subsection describes the evolution of the hardware implementation, in terms of design accuracy and performance metrics. It details the several iterations made to reach the optimized design described in Section 3.

4.3.1. Initial 2x5 Neuron Array with SCW

The initial design for this system included the delta modulation of ECG signal, serial scan chain wrapper encompassing the 2x5 neuron array, and WTA circuit. The neuron array included instantiations of the synapse unit, accumulator, and compare and fire

modules. However, this initial design revealed several bugs or undesirable results in simulation; some of the initial architecture implementation choices are described below.

This initial design used the testbench to integrate the delta modulated spike input into the rest of the system. The compare and fire module was separate from the accumulator module and served to take in the membrane potential as an input and evaluate it against a threshold to fire the output neuron action potential; this module used various internal states and rising edge detection logic blocks to perform the simple comparison task. The initial implementation of the delta modulator induced a constant ‘high’ signal during significant increases or decreases in the ECG signal, as opposed to pulsed spike streams. The first WTA implementation was not done modularly; it was implemented sequentially in the neuron array module. Additionally, it used 3 bits to represent the 5 output classifications, as opposed to one-hot encoding, like the column fire signal. The combination of these initial design choices, as well as the pipeline of data flow through the synapse unit and accumulator modules, introduced various bugs – 2 cycle fire signals (as opposed to 1), extra cycles of accumulation in all columns leading to incorrect fires, latched WTA winner outputs, and incorrect STDP updates.

The timing diagram, shown in Fig 4.1 below, demonstrates some of the functional and nonfunctional attributes of the initial system. Briefly put, the testbench initializes the system and begins processing ECG data; a more comprehensive description of the simulation is described in Section 5.1.2. The output of the delta modulator is ‘spike_in[1:0]’; the spike streams are constant, and the presence of a spike triggers the ‘weight_to_accum’ or ‘syn_out’ values from the synapses units to be accumulated by the accumulators, shown in the ‘accum_cols’ result. This initial design did not accumulate

values into the negatives. The basic data flow through the system is correct; the input spikes trigger synaptic stimuli, which is integrated, fires when a column exceeds the threshold, and is classified by the WTA ‘winner’ signal. The membrane potential accumulates the cycle after a spike triggers the synaptic stimuli in the synapse unit. Furthermore, the STDP is evident in ‘syn_out[1][1]’, as the proximity of the input spike to the output spike triggers an increase in that weight, from 2, at 355ns, to 4, seen at 425ns. However, there are extra cycles where accumulation can occur before the membrane potential is reset and the other columns are inhibited by the WTA. In certain cases, this would trigger incorrect action potentials from other columns. Additionally, the synapse unit and accumulators perform calculations, like determining the next weight to pass and adding empty values to the stored accumulated value, when there are no spikes and the calculation is unnecessary. Unnecessary arithmetic can cost area, use dynamic power, and create a longer critical path.

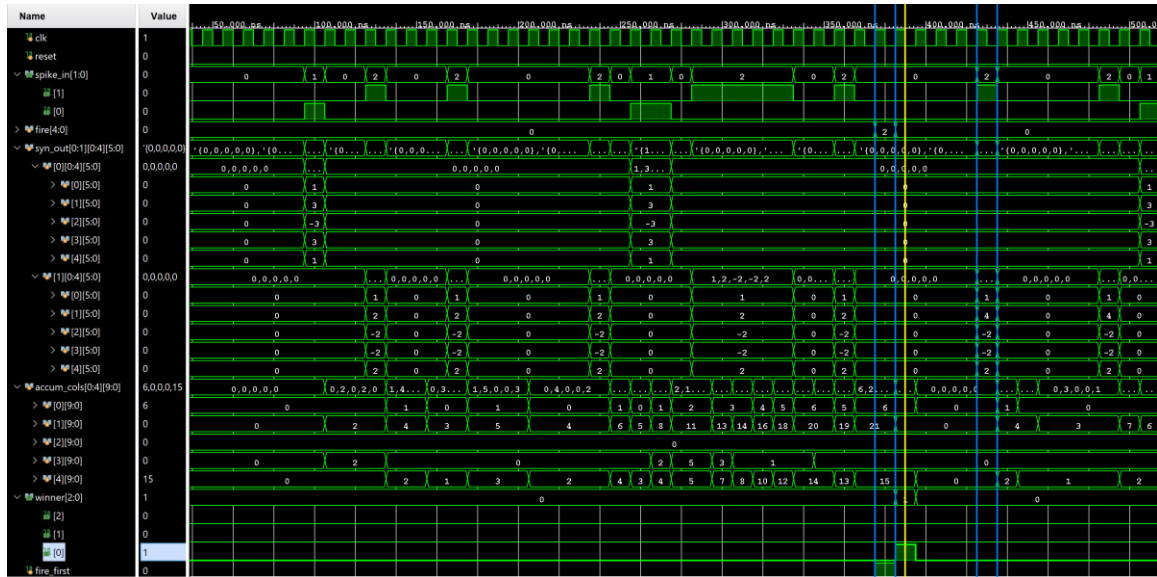


Fig. 4.1: Initial Neuron Array Timing Diagram

This initial system implements the SCW to serially read the program data, passed in from the testbench, and parallelly load it when ‘load_config’ pulses high. The 10 initial synaptic weights, fire threshold, refractory period, and leak rate are the system parameters the SCW passes in. The timing diagram shown in Fig. 4.2 reflects the parallel loading of the system parameters at 785 ns.

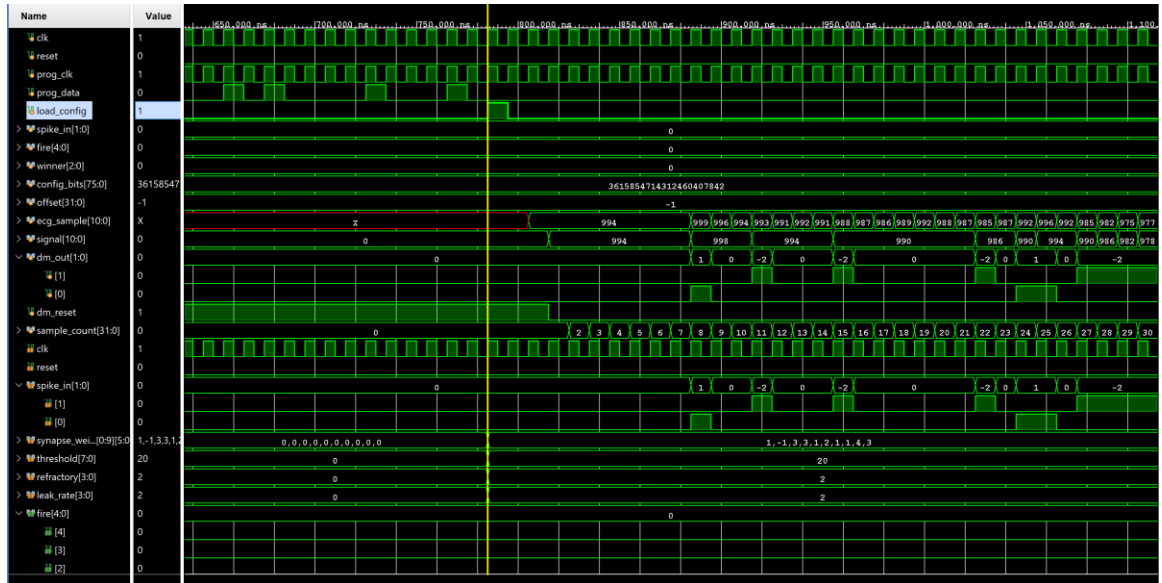


Fig. 4.2: Scan Chain Wrapper to System Parameter Timing Diagram

Additionally, to explore the timing resiliency of the design and the consequent iterations, the post synthesis/implementation WNS at 100MHz were evaluated on 3 FPGA targets to estimate the maximum operating frequency. The Basys3 yielded an estimated maximum frequency of 185MHz. The Spartan-7 yielded an estimated maximum frequency of 181MHz. The Kintex 7 yielded an estimated maximum frequency of 285MHz. These numbers were at the higher end of the expected maximum operating frequency and provided a baseline for improvement. All 3 FPGA implementations yielded modest utilization numbers: 768 LUTs and 438 FFs. Ultimately, the initial baseline model

establishes the data processing pipeline from ECG input to output classification with the intended learning dynamics and runtime reconfigurability; however, some of the shortcomings in the design accuracy results promoted the following changes:

- Change the delta modulated spike streams continuous activation portions to be a pulsed bitstream to maintain the desired spiking nature.
- Add logic to utilize the enable signal of each module (synapse unit and accumulator) to ‘turn on’ once SCW parallel load is complete.
- Ensure the synapse unit and accumulator properly perform the gated accumulation without performing extra computations.
- Integrate the ‘fire’ computation (evaluating the accumulated value against the threshold) into the accumulator module.
- Implement the WTA modularly and with combinational logic; one-hot output classification as opposed to bitwise representation.

4.3.2. Improving Design Accuracy and Metrics

The first changes that were made had to do with the relative timing between the accumulated value, fire action potential, and WTA classification. By integrating the compare and fire computation in the same module as the accumulator, the rising edge detection logic and sequential ‘fire’ signal assignment could be simplified. Initially, fire was implemented combinational based on the sequential accumulated register value, to remove the extra cycles of accumulation and unwanted latency when firing, resetting, and inhibiting the other columns. Additionally, the WTA was implemented as a module and combinational based on the ‘fire’. The timing diagram in Fig. 4.3 demonstrates the clean

coordination between reaching the threshold, firing, and resetting membrane potential after the winner was chosen. Furthermore, the enable signals were utilized on all neuron array modules from the top scan chain wrapper to only turn on the array once the system parameters were processed. This removes any incorrect and unnecessary computations from occurring while the system is still processing the data and waiting for the configuration signal.

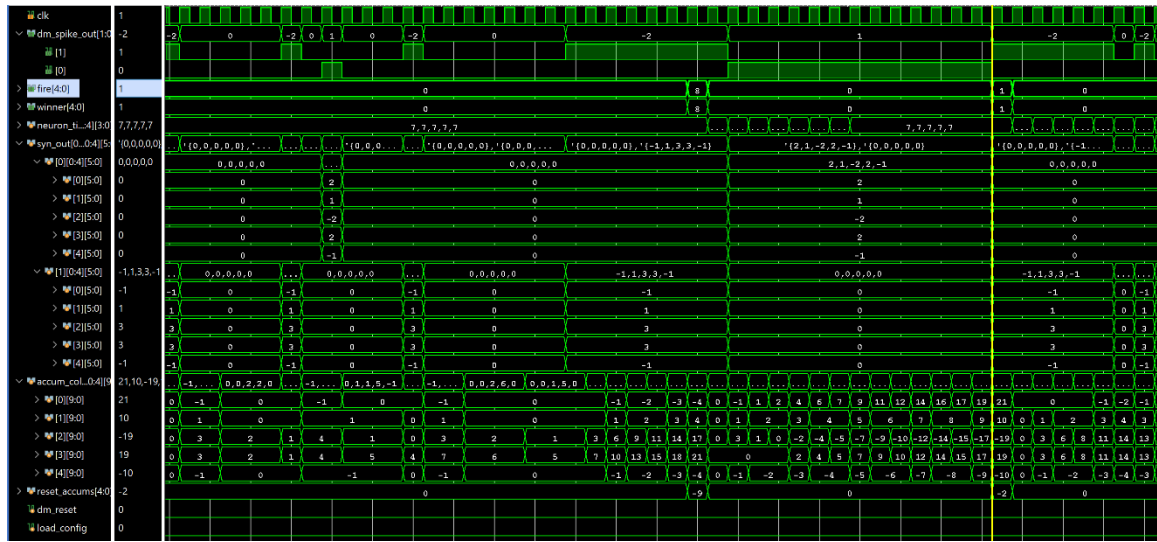


Fig. 4.3: Second Iteration Neuron Array Timing Diagram

Table 4.1: First Accurate Design Metrics

FPGA Target	WNS @ 100MHz (ns)	Estimated Maximum Frequency (MHz)
Basys-3	0.576	106
Kintex 7	2.964	142

The first set of changes made significantly improved the design accuracy of the system, especially with respect to the relative timing of the spike propagation from input to output. However, Table 4.1 demonstrates that these changes had a negative effect on the estimated maximum frequency of the system. Additionally, the STDP calculation is not

carried out as intended during long streams of ‘high’ ECG input compared to portions of pulsed input spikes. Table 4.1 reflects the updated timing metrics for the entry-level Basys-3 and higher performance Kintex-7 boards. Ultimately, these changes moved the system closer to the desired operation, while increasing the length of the critical path.

To make the linearly approximated STDP weight update calculations more consistent and accurate, long streams of ‘high’ input spike trains from the delta modulator were changed to be pulsed. Additionally, the logic in the accumulator and synapse unit was modified to help break up the long critical path that stretched from the output of the delta modulator (DPE input spike streams) to the accumulated membrane potential register; pipelining the computation that accumulates the synaptic stimuli was one strategy used to improve the timing margin that was an acceptable design accuracy choice. Exploring the effects on the timing margin and design accuracy when changing the key signals in the critical path from combinational to sequential, or vice versa, and how to register that value to pass to other modules enabled the improvement of both factors. These signals include ‘delta_t’, ‘lut_index_reg’, and ‘stdp_calc_reg’ from the synapse unit and ‘accum_reg’, ‘pre_accum’, and ‘fire’ from the accumulator.

This version maintained the required design accuracy, while significantly improving the previous drop in performance metrics. The delta modulator input spike train is now pulsed, shown by the insertion of low values in ‘dm_spike_out’ between the long stream of activity from the delta modulator in Fig. 4.4. Additionally, signals to determine if the STDP calculation should be valid, ‘stdp_calc’, and indexing into the STDP LUT, ‘lut_index’, are calculated combinational, but are sequentially registered for any updates on the clock edge; the ‘pre_accum’ signal to add the two synapses in a column in the

accumulator is changed from combinational to sequential as well. This introduces an extra cycle of latency between the synaptic stimuli or weight to be accumulated and the execution of the integration to the membrane potential. However, it was decided that this is still an accurate way to process the data, since no values are missed and neuron leaking and refractory period still performs as desired. Ultimately, this iteration of the design established the baseline for what the accurate design is and began to improve on the timing and resource utilization metrics.

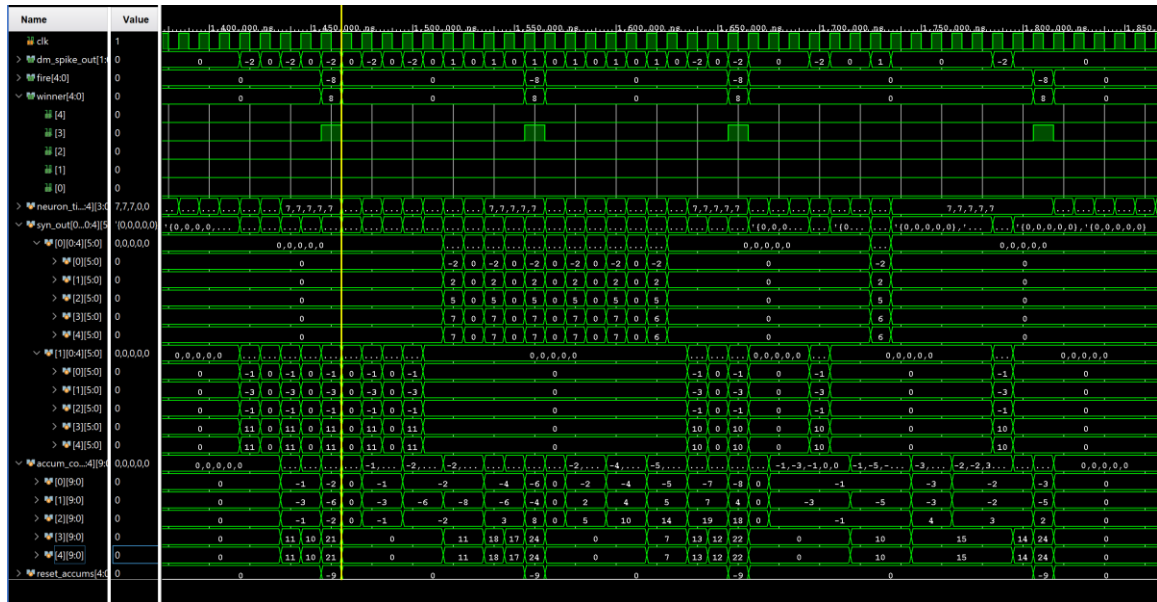


Fig. 4.4: Third Iteration Neuron Array Timing Diagram

Table 4.2: Intermediate Accurate Design Metrics

FPGA Target	System Operating Frequency (MHz)	WNS (ns)	Estimated Max. Frequency (MHz)	LUT Count (% Util)	FF Count (% Util)
Basys-3	143	0.813	161	549 (2.64)	419 (1.01)
Kintex 7	200	0.766	236	548 (1.34)	419 (0.51)

Table 4.2 reflects the improved metrics for this iteration; the estimated maximum frequency has improved to almost return to the initial values, before improving the design

accuracy. Additionally, the resource utilization numbers dropped significantly: the number of LUTs used dropped by over 200, and the number of FFs dropped by 19. The final iteration continues to improve the timing metrics, resource utilization, and dynamic power consumption for FPGA implementation, while maintaining accuracy and preparing the RTL for the ASIC flow.

4.3.3. Final Metric Improvements

To continue to maximize the potential operating frequency of this system, the critical path had to continue to be identified and shortened after each improvement. Additionally, it was important to identify how much of the total delay of the critical path was logic/propagation delay or net/wire delay. The following changes were made to further decrease the length of the critical path and improve the potential maximum operating frequency.

- Changing the ‘fire’ computation from combinational to sequential and killing it after one cycle, while maintaining the logic to determine whether accumulation should occur as combinational. The result was a significant improvement in the timing margin, while maintaining no extra accumulations and a proper reset.
- Setting a max fanout constraint of 4 to the signals that add the synaptic stimuli and store the membrane potential to decrease the net delay in the critical path.
- Explicitly performing repeated computations as wires.

The timing diagram in Fig. 4.5 reflects an accurate design, with the correct communication via spikes throughout the neuron array, from delta modulator output to WTA classification. The synapses effectively pass the gated synaptic weight contributions

based on the input spikes streams to the accumulators and perform its STDP computation and the accumulator captures and accumulates all contributions, fires when the threshold is exceeded and resets. The complete description of the final, optimized architecture implementation is described previously, in Section 3.1.



Fig. 4.5: Fourth Iteration Neuron Array Timing Diagram

Table 4.3: Best FPGA Implementation Performance Metrics

FPGA Target	System Operating Frequency (MHz)	Estimated Max. Frequency (MHz)	LUT Count (% Util)	FF Count (% Util)	Dynamic Power Consumption (mW)
Basys-3	200	211	509	424	0.032
Arty-A7	200	207	516	424	0.043
Spartan-7	200	215	514	424	0.042
Kintex 7	300	324	516	424	0.061

The result of many iterations of architectural and logic improvements is summarized by Table 4.3; these results proved to be the penultimate FPGA Implementation metrics, as a few small changes were made to ensure high design accuracy; a final analysis of the full, end-to-end system revealed a few bugs and shortcomings in this version; the

correcting of these issues would slightly worsen the best FPGA Implementation metrics shown in Table 4.3, but lead to a final, design accurate, optimized design. The final changes that were made are described modularly, as follows:

- Synapse Unit: Modify the ‘stdp_calc’ signal to only depend on the neuron time and synapse times; previously considered the refractory period. Additionally, ensure the proper indexing into the LUT.
- Accumulator: Fix the extra cycle of refractory period that previously occurred. Parameterize the number of input synapses, to allow for scaling of DPE array size.
- Delta Modulator: Change the threshold from a parameter to an input, to allow initialization and reconfigurability from the SCW.
- Neuron Array: Factor in the parametrized input synaptic weights to accumulator from the synapse unit.
- Scan Chain Wrapper: Include the delta modulator threshold as a system parameter, to be serially processed.

The final, optimized version of the system, described previously in Section 3, demonstrates the potential for high frequency operation in the context of a neuromorphic system, with low dynamic power consumption and resource consumption. It is a product of the iterative process of improving design accuracy and performance metrics. The final FPGA Implementation results and subsequent analysis are described in Section 5.1.

4.3.4. Changes for RTL to GDSII Flow

Further modifications were made to the design to be able to successfully pass the automatic RTL to GDSII flow while maintaining the same logic and structure. These changes included the flattening of 2D arrays into 1D array with bit slicing and the removal of any inferred latches that weren't previously caught. A more detailed description of the ASIC implementation flow is provided in Section 5.2.

5. FPGA AND ASIC IMPLEMENTATIONS

This section details the hardware results of the proposed architecture, through FPGA and ASIC implementation. It details the methodology behind verifying and implementing the design in both modes and analyzes the relevant results. For the FPGA implementation, the main metrics explored are timing, resource utilization, and power. For the ASIC implementation, the primary metrics are power, performance, and area (PPA).

5.1. FPGA Implementation

FPGA implementation was conducted by running synthesis and implementation on the hardware design in Vivado on 4 FPGA development boards. A detailed description of the verification methodology, behavioral simulation, and FPGA implementation results follows.

5.1.1. Verification Methodology

The system was verified using a simulation-based methodology implemented in SystemVerilog, covering both unit and system-level requirements. The top-level testbench was constructed around the scan-chain wrapper (SCW) to serve as the complete system entry point, covering the scan-chain programming, delta modulator, DPE-based SNN, and WTA circuit. The testbench drives the system using real ECG samples from the MIT-BIH Arrhythmia Dataset. The functional correctness was verified by inspecting that the internal signals at every hierarchical level of the system behave as expected in the context of the full-signal processing pipeline. The following subsection describes the top-level testbench used to verify the behavior of each unit in the system.

5.1.2. Behavioral Simulation - Top-Level Testbench around SCW

The top-level testbench simulates the entire system as one Device Under Test (DUT); the SCW instantiates every lower-level module, which allows driving the top-level ports to be sufficient to verify the end-to-end functionality of the entire system. Furthermore, it validates both the scan-chain programming path, as well as the signal-processing path from ECG input to WTA classification in the same simulation. By utilizing the scope in the waveform view on Vivado, each lower-level DUT instantiation was able to be analyzed.

Additionally, the testbench mirrors the RTL parameterization, featuring 2 input rows, 5 neurons per row (output columns), 6-bit synaptic weights, 8-bit accumulator and threshold, 4-bit delta modulator step size and 3-bit timing counters. Since the serial shift is done in a bitwise manner, the 'TOTAL_BITS' parameter sums all the scan chain fields, allowing the system parameters to automatically scale for varying DPE dimensions.

The sequence of utilizing the scan chain programming is carried out as follows. First, the local parameters, integers, and logic were initialized and the SCW DUT was instantiated. The testbench utilizes two clocks: a system clock, and program clock. This allows for two separate clock domains/frequencies to avoid metastability and a slower programming clock if required. However, they are run at the same speed (100 MHz) since there is no concurrent access conflict between the two domains during normal operation. A task, 'serial_in', was used to serially shift a 'TOTAL_BITS' amount of program bits by the MSB first on every positive edge of the program clock. The program bits are prepared into an array to be passed on 'serial_in'; this data is 10, random, non-zero values between -3 and 5 for the arbitrary, initial synaptic weights, a threshold value of 20, a refractory

period of 2 cycles, leak frequency of every 4 cycles, and delta modulator step size of 4. After the serial shift task is complete, 'load_config' is pulsed on the system clock to capture the configuration to the functional registers. This signal is used in the SCW DUT to signify that the parallel load of system parameters is ready to be performed, and the system is initialized and ready for input spikes.

To provide the 11-bit ECG sample input to the system, the testbench first opens a text file containing the discretized samples from the MIT-BIH Arrhythmia Database. Once the file is loaded, the delta modulator reset drops, and the data is read sequentially, and a sample is applied on every system clock cycle. Additionally, there are bounds to ensure that the read sample is within 11 bits, or ranges from 0 to 2047. There is significant timing headroom, as the system clock (100 MHz) runs significantly faster than the sampling rate of the data (360 Hz).

The observability at the top level focused on the end-to-end validation of the system, as it can monitor the ECG sample stream to the delta modulator, 5 column fire action potentials, and WTA classification. However, the lower-level modules instantiated by the SCW can be added to the waveform to confirm the functionality of each part of the pipeline, as shown in Section 3.

The simulation waveform in Fig. 5.1 shows the relevant signals of the testbench design unit, similar to those in the SCW DUT waveforms. The 'prog_clk' is used to pass data to the SCW DUT, and 'load_config' is pulsed once the scan is completed. This triggers the reset of the delta modulator and enables the neuron array, which begins to receive the 'ecg_sample' read from the text file.

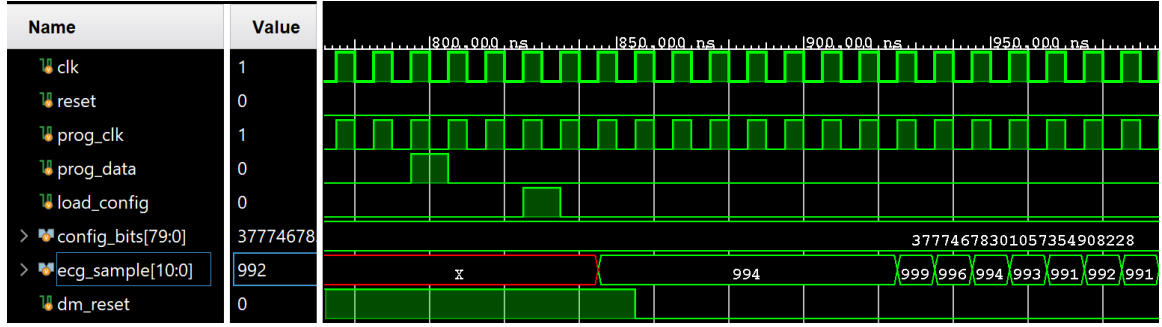


Fig. 5.1: SCW Testbench Simulation Waveform

5.1.3. FPGA Results

Post synthesis and implementation results were gathered for 4 FPGA development boards spanning two families in the Xilinx 7 series [49]: Basys3 (XC7A35T), ArtyA7 (XC7A100T), Spartan-7 (XC7S50), and Kintex-7 (XC7K70T). All 7 series devices share a unified architecture built on the same 28nm high-performance low power (HPL) process technology, utilizing identical 6-input LUT-based configurable logic blocks, block RAM structures, and clock management resources. The architectural consistency ensures that differences in implementation results across boards reflect the device capacity and speed grade. The Spartan-7 and Basys3 (Artix-7 35T) represent the low cost, low power tier targeting resource-constrained applications. The ArtyA7 uses the larger Artix-7 100T, providing about three times the logic capacity of the 35T to potentially reduce routing congestion in the same performance tier. The Kintex-7 extends to a higher performance device and establishes the operating frequency and power ceiling for this architecture. Ultimately, the range of these 4 FPGA development boards provides the means to evaluate the scalability, resource efficiency, timing performance, and power characteristics of the proposed neuromorphic system. The same RTL source was used across all four targets without modification, confirming the portability of the design and supporting its

subsequent transition to ASIC implementation. The subsequent analysis shows that the small logic footprint, linear power-frequency relationship, consistent timing across devices, lack of IP dependencies, and clean parameterization of this system all promote the transition to the ASIC implementation flow, detailed in Section 5.2.

5.1.3.1. Frequency Analysis

The timing performance was analyzed across the 4 FPGA boards; all boards were implemented using the -1-speed grade, to provide a conservative baseline for the timing results. The timing analysis was performed by sweeping the global clock frequency and recording the worst negative slack (WNS) until the system was close to timing failure (less than 0.5ns WNS); this provided a close estimate of the maximum operating frequency of the system for each board, as well as the slack versus frequency graph shown in Fig. 5.2.

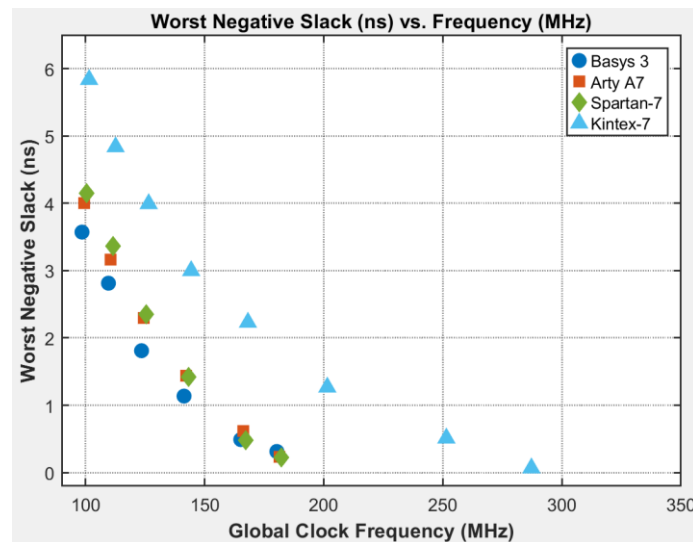


Fig. 5.2: Timing Margin Analysis across 4 FPGA Boards

The maximum operating frequency of the system for each FPGA board was estimated by using the WNS at the highest frequency tested and Equation 5.1. The Basys3,

Arty A7, Spartan-7, and Kintex-7 had a maximum operating frequency of 192.8 MHz, 189.9 MHz, 189.7 MHz, and 291.3 MHz, respectively. The WNS at each frequency, and maximum operating frequencies of the three cost-optimized devices cluster together closely, demonstrating similar intrinsic logic and routing delays for boards with similar speed grades and fabrics. The Kintex-7 shows faster internal routing and optimized interconnect structures compared to the other 3 boards, reflected in the significant separation in its timing performance. This result is expected, as the same RTL is supposed to achieve substantially higher frequencies on the performance-optimized fabric. Additionally, the WNS curves are nearly linear with inverse frequency or period, indicating the critical path is consistent and the design doesn't hit any routing congestion walls as the frequency increases. The maximum frequency headroom over the sampling rate of ECG signal (360 Hz) supports aggressive voltage/frequency scaling to achieve ultra-low power consumption, as well as increasing network density.

$$F_{MAX} = (T_{ACTUAL} - WNS)^{-1}$$

Equation 5.1: Estimated Maximum Frequency

5.1.3.2. Dynamic Power Analysis

The dynamic power analysis of the end-to-end architecture demonstrates a highly efficient linear scaling profile across all 4 FPGA platforms, reflected in Fig. 5.3. All boards cluster closely, starting at about 20mW at a 100 MHz clock, and scaling linearly up to its maximum testing frequency. The low power envelope and high-frequency capability of the system allow for aggressive frequency scaling to minimize power consumption, supporting its potential use in battery powered wearable devices.

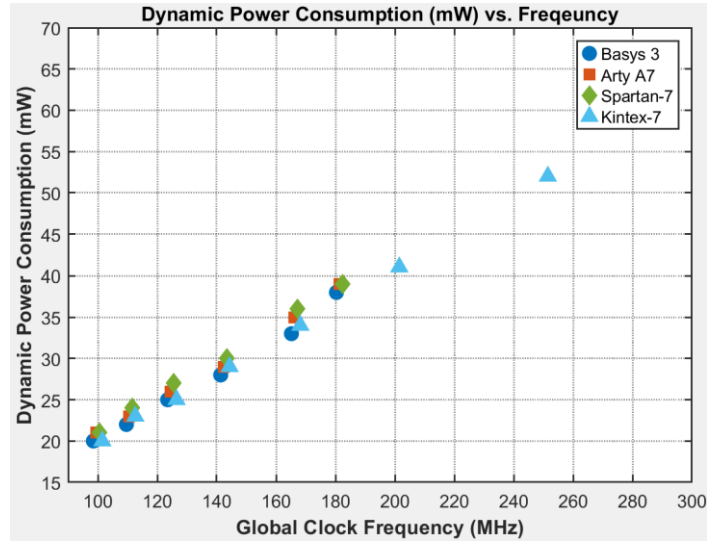


Fig. 5.3: Dynamic Power Consumption Analysis across 4 FPGA Boards

5.1.3.3. Resource Utilization Analysis

The design synthesized and implemented across 4 FPGA boards demonstrates a high resource efficiency; the entire system requires 548-549 LUTs and 432 FFs, revealing an effectively constant logic footprint. This confirms that the architecture is synthesized deterministically and the resource usage is independent of device size, revealing a portable design.

Table 5.1: Resource Utilization Analysis Across 4 FPGA Boards

Resource Metric	Basys3	Arty A7	Spartan-7	Kintex-7
LUT	548	548	549	549
- # Available	20800	63400	64000	41000
- % Utilization	2.63	0.86	0.86	1.34
FF	432	432	432	432
- # Available	41600	126800	128000	82000
- % Utilization	1.04	0.34	0.34	0.57
IO	27	27	27	27
- # Available	106	210	400	285
- % Utilization	25.47	12.86	6.75	9.47

Table 5.1 summarizes the resource utilization results across all 4 FPGA boards. The Basys3 consumes the highest percent utilization at 2.63% LUTs and 1.04% FFs due to it having the smallest fabric; the rest of the boards exhibit much lower resource utilization. The small footprint promotes the scaling of the DPE array or cascading multiple DPE arrays. The use of the SCW reduces the I/O count of the system to 27, which is the highest utilization percent on the Basys3 at 25.47%. The resource utilization analysis shows a compact architecture suitable for the smallest, cheapest FPGAs.

5.2. ASIC Implementation

An ASIC implementation of the hardware system was done to evaluate the power, performance, and area (PPA) of this design. A description of the RTL-to-GDSII flow and the ASIC implementation results follows.

5.2.1. RTL to GDSII Flow

The ASIC Implementation was carried out using Librelane/OpenLane 2, a fully open-source automated RTL-to-GDSII flow infrastructure library which enables the construction of digital ASIC implementation flows based on open-source and commercial EDA tools [50]. This includes Yosys for synthesis, OpenROAD for floorplanning, placement, clock tree synthesis, and routing, Magic and KLayout for physical verification, and Netgen for layout-versus-schematic (LVS) checks [51]. The flow executes the entire physical implementation sequence – synthesis, floorplan initiation, I/O placement, power distribution network generation, global/detailed placement, clock tree synthesis (CTS), global/detailed routing, parasitic extraction, static timing analysis (STA), and design rule check (DRC)/LVS signoff – in a single automated pass through the use of a configuration JSON file. The target technology is the openly available SkyWater SKY130 (SKY130A) process node [52]. It is widely adopted for academic and research purposes for prototyping low-power digital and mixed-signal designs. The combination of the open-source flow and PDK enabled the implementation of this system.

To conduct the flow, the config.json file, shown in Code Block F, defines the required variables, as well as some optional variables [53]. The design name, Verilog files, clock period in nanoseconds, clock net name for CTS, and clock port name for STA are all

required configuration variables. The synthesis strategy (SYNTH_STRATEGY) was set to 'AREA 0', directing the synthesis tool to prioritize minimizing area during technology mapping. This aligns with the goal to minimize the silicon footprint for potential wearable deployment. The floorplan core utilization (FP_CORE_UTIL) was set to 30% with a placement target density (PL_TARGET_DENSITY_PCT) of 35% to provide enough whitespace for the router to place interconnects without congestion, while keeping the die area compact. A maximum fanout constraint (MAX_FANOUT_CONSTRAINT) of 8 was applied to limit the number of sinks driven by any single cell. This will reduce the net capacitance and improve the timing of high-fanout signals. The remaining configuration parameters address antenna rule violations. Additionally, heuristic diode insertion was enabled to insert protective diodes on long metal segments that could accumulate charge during fabrication and damage gate oxides.

```

{
  "DESIGN_NAME": "scan_chain_wrapper",
  "VERILOG_FILES": [
    "dir::scan_chain_wrapper.sv",
    "dir::neuron_array.sv",
    "dir::delta_mod.sv",
    "dir::accumulator.sv",
    "dir::synapse_unit_2.sv",
    "dir::wta.sv"
  ],
  "CLOCK_PERIOD": 10.0,
  "CLOCK_PORT": "clk",
  "CLOCK_NET": "clk",
  "pdk::sky130A": {
    "FP_CORE_UTIL": 30,
    "PL_TARGET_DENSITY_PCT": 35,
    "MAX_FANOUT_CONSTRAINT": 8,
    "SYNTH_STRATEGY": "AREA 0",
    "GRT_REPAIR_ANTENNAS": true,
    "RUN_HEURISTIC_DIODE_INSERTION": true,
    "DIODE_INSERTION_STRATEGY": 3,
    "GRT_ANTENNA_ITERS": 10,
    "DPL_RELAX_DIODE_PLACEMENT": true
  }
}

```

Code Block F

5.2.2. ASIC Results

Table 5.2: ASIC Implementation Metrics

Clock Frequency (MHz)	1	100	200
Design Instance Area (μm^2)	31,647.9	31,799.2	33,941.3
Design Instance Count	4478	4483	4552
Actual Core Utilization (%)	37.11	37.29	39.80
Total Power (mW)	0.07036	7.134	14.68
Switching Power (mW)	0.02623	2.638	5.617
Setup Worst Slack (ns)	794.35	1.2534	-2.2242

The process produced a LVS and DRC clean layout, shown in Fig. 5.4, while meeting a target frequency of 100 MHz. In addition to evaluation at 100 MHz, timing, power, and area tradeoffs were explored at lower and higher clock frequencies of 1 MHz and 200 MHz; the timing, area, and power results are summarized in Table 5.2. For all three implementation runs, the design die area was $95,792.5 \mu\text{m}^2$ and the design core area was $85,280.5 \mu\text{m}^2$.

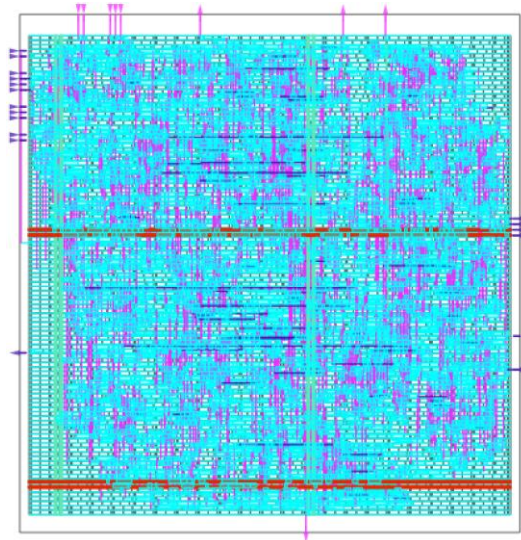


Fig. 5.4: ASIC Implementation Layout at 100MHz

The design comfortably meets the timing requirements at 100MHz and is vastly over positioned at the lower 1 MHz frequency; however, timing fails at 200 MHz, placing the estimated maximum frequency for this ASIC implementation at around 114 MHz. While this is lower than the maximum frequency achieved on the FPGA targets, it can be attributed to less mature and less speed-optimized tools and SCL and is still more than sufficient for the target application.

The die and core area remain fixed across all three clock frequency implementations. The design instance area, instance count, and core utilization stay

relatively constant at 1 MHz and 100 MHz, while increasing modestly at 200 MHz. This may be due to the synthesis tool attempting to close the aggressive timing constraint by using larger, faster drive-strength standard cells or adding buffers. However, the large footprint and large portion of core area available promotes the scaling of the DPE array.

Both the total power and switching power scales linearly with frequency, which confirms the expected relationship between power and frequency. The ASIC implementation power consumption boasts a significant reduction from the FPGA implementation, demonstrating the gains from eliminating the programmable interconnect overhead of the FPGA fabric. Additionally, the already low power consumption can be aggressively scaled towards and potentially past 1 MHz to bring dynamic power consumption to the microwatt range. However, the 100 MHz design could better take advantage of the benefits of the sparsity of spikes in a neuromorphic system.

6. SOFTWARE MODEL

This section describes the software model used to evaluate the ECG classification accuracy of our system. The software model aims to recreate the DPE based SNN system, using PyTorch and the `snnTorch` libraries. Training is done using built in functions, and testing and evaluation is done with a custom software implementation of the hardware system. This section details the background of `snnTorch` and the functions used, the method behind implementing and training the software model, and the results of testing the software replication of the hardware model under different circumstances.

6.1. `snnTorch` Introduction

`snnTorch` is a Python package for performing gradient-based learning with spiking neural networks [54]. It integrates pre-designed spiking neuron models within the PyTorch framework, which can be treated as recurrent activation units. Essentially, this open-source Python library enables the building and training of SNNs by treating spiking neurons as standard, interchangeable activation layers in a neural network.

6.1.1. LIF Neuron

As prefaced in Section 2.4.1, the LIF neuron takes the sum of weighted inputs and integrates it over time with leakage, as opposed to passing it directly to an activation function; it emits a voltage spike when the integrated value exceeds the threshold. By abstracting away the shape and profile of the output spike, information is now stored in the timing of spikes, not the spike itself.

`snnTorch` currently supports several implementations of the LIF neuron; this implementation utilizes the first-order LIF neuron model, ‘`snntorch.Leaky`’ [55]. It

assumes a current injection as input, and uses a decay rate, β , to control how quickly the membrane potential leaks; β is treated as a hyperparameter. At each timestep, the membrane potential decays by a factor of β , incoming current is added, and if the neuron fired, the threshold is subtracted (reset by subtraction); the ‘subtract’ reset is a biologically plausible method, as it preserves any excess charge above the threshold. This combination of decay, input, and reset is the recursive representation of the LIF neuron, described by Equation 6.1. Additionally, the neuron manages its own membrane state internally; the state is cleared between samples, so the membrane potential doesn’t carry over.

$$U[t + 1] = \underbrace{\beta U[t]}_{\text{decay}} + \underbrace{WX[t + 1]}_{\text{input}} - \underbrace{R[t]}_{\text{reset}}$$

Equation 6.1: recursive representation of a LIF neuron [56]

6.1.2. Training SNNs and Surrogate Gradients

The formulation of the spiking neuron in a discrete, recursive form is suited to take advantage of methods in training recurrent neural networks (RNNs). Shown in Fig. 6.1, the unrolled graph provides an explicit description of how the computations are performed. The information flows forward in time to compute the outputs and losses, and backwards in time to compute gradients.

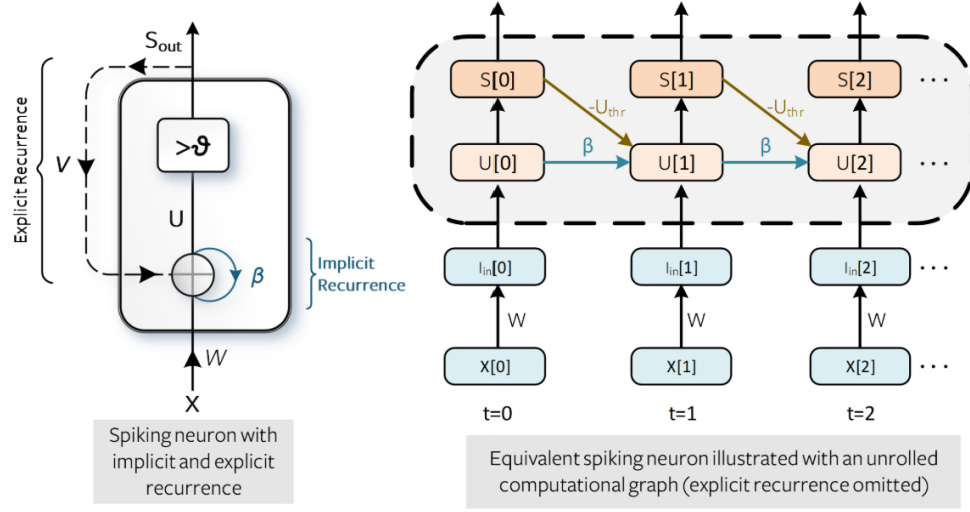


Fig. 6.1: Recurrent Representation of Spiking Neurons [56]

The goal is to train the network using the gradient of loss with respect to the weights; backpropagation achieves this using the computation detailed in Equation 6.2 and Fig. 6.2. However, the gradient will almost always be nulled to zero, since the derivative of the Heaviside step function is a Dirac Delta function, which evaluates to 0 everywhere except at the threshold, where it is infinity. This is known as the dead neuron problem [56]. Since the spikes are not differentiable, a surrogate gradient replaces the true derivative during backpropagation [57]; ‘fast_sigmoid’ is used as the surrogate in this implementation. This allows the forward pass to still produce binary spikes, while the backward pass is estimated. A depiction of using surrogate gradient descent to solve the dead neuron problem is shown in Fig. 6.2.

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial S} \underbrace{\frac{\partial S}{\partial U}}_{\{0, \infty\}} \frac{\partial U}{\partial I} \frac{\partial I}{\partial W}$$

Equation 6.2: Backpropagation Algorithm Chain Rule Equation

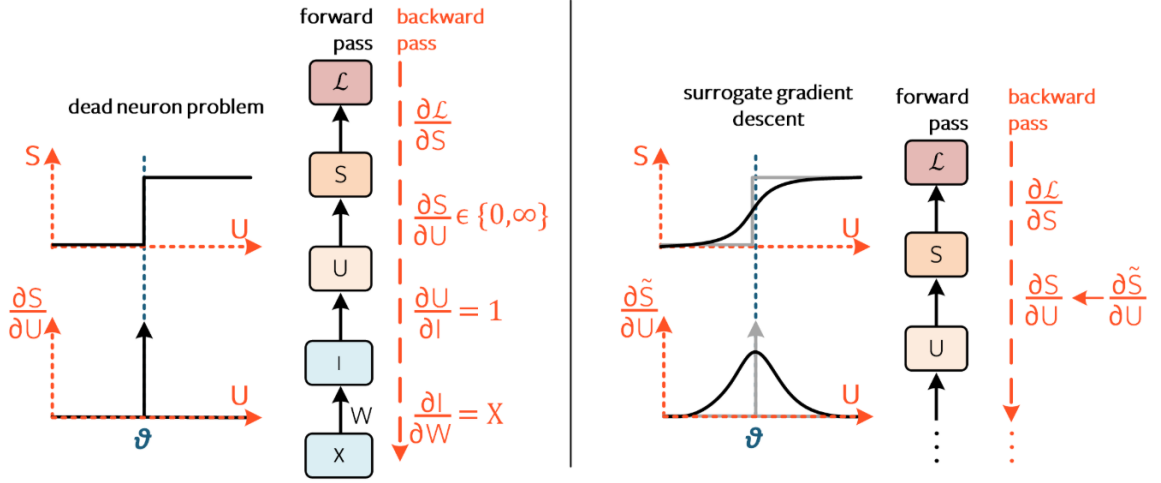


Fig. 6.2: Spike Non-differentiability and Surrogate Gradient Descent [58]

However, Equation 6.2 only calculates the gradient for a single time step, or the immediate influence. The back propagation through time (BPTT) algorithm is used to calculate the gradient from the loss to all descendants, and sum them together.

6.1.3. Loss Function

In general, loss functions are used in machine learning to measure how far the algorithm's predictions are from the actual values; the goal is to minimize loss. 'snntorch.functional' implements common arithmetic operations applied to spiking neurons, such as loss and regularization functions. The training in this model uses cross entropy spike count loss, which encourages the correct class to fire at all time steps, and aims to suppress incorrect classes from firing [59]. It does so by applying standard cross-entropy loss to the single spike count per output neuron per sample. This operation is suited for the architecture, as the classification decision is based on the total spike count via WTA, as opposed to per-timestep outputs. Furthermore, the loss function accepts a weight

parameter that scales the penalty for each class, inversely proportional to its frequency in the training set. This can help mitigate class imbalance in a dataset and will be discussed further in Section 6.2.

6.2. Model Architecture and Results

This section describes the software training model used to develop and evaluate the initial synaptic weights for the neuromorphic hardware classifier representation. Starting from baseline implementation, a series of modifications to weight quantization and mapping, loss function design, and training parameters are introduced and analyzed. The results are evaluated on the hardware-faithful simulation at each stage to assess their impact on classification performance across all five heartbeat classes.

6.2.1. Baseline Implementation and Results

The built-in `snnTorch` modules are utilized for training, while testing is done on a Python simulation, which mirrors the RTL hardware design; the trained synaptic weights are passed to the test module. The MIT-BIH Arrhythmia Dataset is split equally between training and testing ECG recording excerpts [60] and is classified based on the AAMI classification convention.

The `DPE_Train` model uses a fully single connected layer – `nn.Linear` with 2 inputs, 5 outputs, and no bias – feeding into `snn.Leaky LIF` neurons. The `DeltaMod` converts quantized ECG windows into two-channel spike trains, replicating the hardware implementation exactly. The training model processes 200 timesteps per heartbeat, the LIF neurons accumulate the weighted current and decay by β (no refractory period or

STDP), and the classification is based on which output neuron spikes the most per heartbeat. The initialization of the DPE_Train class is shown in Code Block G.

```
class DPE_Train(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(2, 5, bias=False)
        self.lif = snn.Leaky(beta=0.95,
                              spike_grad=surrogate.fast_sigmoid(),
                              init_hidden=True)
```

Code Block G

The HW_Net class attempts to replicate the RTL design in Python and uses the trained weights from DPE_Train as the initial synaptic weights of the system. The Synapse class models the hardware synapse, with integer weights and optional STDP learning via look-up-table. The Neuron class implements the hardware LIF representation with the accumulation, digital leaking with a counter, configurable threshold, and a refractory period. The WTA enforces lateral inhibition by resetting all non-winning neurons when any neuron fires. Classification is achieved in the evaluate function using a similar spike-counting approach as the training model by predicting the neuron that won the WTA classification the most over the 200 timesteps; the evaluate function is responsible for generating the delta modulated spikes to be passed to the HW_Net at each timestep. The initialization of the HW_Net is shown in Code Block H.

```

class HW_Net:
    def __init__(self, weights, stdp=False):
        self.stdp = stdp
        self.syn = [[Synapse(weights[i][j]) for j in range(2)] for i in
                      range(5)]
        self.neurons = [Neuron() for _ in range(5)]

```

Code Block H

There are several key differences between the training and testing models: exponential decay versus digital leak counter, no inhibition during training versus WTA inhibition during testing, no refractory period in training, and mapping floating-points trained weights to integer weights for testing. These mismatches mean the weights optimized for the training model are not necessarily optimal for the hardware.

Furthermore, the percent distribution of the 5 classes in the test set is as follows: 89.03%, 3.70%, 6.48%, 0.78%, and 0.014% for classes N, S, V, F, and Q, respectively; The dominance of normal beats makes it difficult to learn weights to classify the minority samples. This is reflected in the training and evaluation of the baseline software model. The raw, trained weights are casted to integer values for the testing model; for evaluating various parameters – threshold, leak rate, refractory period – the accuracy is always 89%. The confusion matrix showing the true positive (TP) prediction reveals the test model classifies everything as Class 0, or normal beats. The confusion matrix shown in Fig. 6.3 shows the initial confusion matrix and classwise accuracy. The combination of unweighted loss, class imbalance, and inadequate weight scaling all contribute to the inability to classify a heartbeat as anything but normal.

```

Test set class distribution:
Counter({0: 44245, 2: 3220, 1: 1837, 3: 388, 4: 7})

Confusion Matrix (rows=true, cols=predicted):
      N      S      V      F      Q
N  44245      0      0      0      0
S   1837      0      0      0      0
V   3220      0      0      0      0
F    388      0      0      0      0
Q      7      0      0      0      0

Per-class accuracy:
N: 44245/44245 = 1.0000
S: 0/1837 = 0.0000
V: 0/3220 = 0.0000
F: 0/388 = 0.0000
Q: 0/7 = 0.0000

Overall: 44245/49697 = 0.8903

```

Fig. 6.3: Baseline Implementation Confusion Matrix and Classwise Accuracy

6.2.2. Weight Quantization

The initial implementation simply casts the floating point, trained weights to integers to map from the snnTorch based training model to the hardware-faithful testing model; the combination of the nature of the training and the fact that int casting in Python truncates towards 0 effectively collapses most weights. To fix this, a scaling approach - mapping floating-point weights to fill the available bit range - can be utilized to better map the trained weights to integers for testing. Additionally, different synapse weight bit widths - 4 bits versus 6 bits - allows the visualization of the necessary dynamic range for the weight differences between classes to survive the quantization to integer value. The scaling from trained, floating-point weights to 6-bit, signed integer weights is shown in Fig. 6.4.

```

... Raw weights:
[[ 0.19712056 -0.06509238]
 [ 0.19521536 -0.05725737]
 [ 0.23675638 -0.3619627 ]
 [ 0.18333988 -0.0063649 ]
 [ 0.22654982 -0.32644004]]

Scale factor: 85.6442

Scaled weights (6-bit signed, range [-32, 31]):
[[ 17  -6]
 [ 17  -5]
 [ 20 -31]
 [ 16  -1]
 [ 19 -28]]

```

Fig. 6.4: Mapping from Trained Raw Weights to 6-bit Signed Scaled Weights

6.2.3. Class-Balanced Training

Since the data set is 89% normal, even with well-scaled weights, the network predicted only normal heartbeats since that minimizes the unweighted loss. Implementing a weighted loss based on the inverse frequency of each class will penalize misclassifying the other, more rare beats more than misclassifying a normal beat. This will discourage the optimizer from always predicting normal heartbeat, since it is no longer the path of least resistance. The tradeoff of being able to achieve multi-class discrimination is a reduction in the overall accuracy.

6.2.4. Evaluation of Improved Model

The implementation of mapping to scaled 6-bit weights and class-balanced weighted loss function enables sufficient class discrimination. Sweeping different threshold values with this improved model allows insight into the balance between normal distributions with lower threshold values and multi-class separation with higher thresholds. For the thresholds of 5, 10, 15, 20, 50, and 100, the improved model shows an overall

accuracy of 81.9%, 75.9%, 66.2%, 59.0%, 59.4%, and 32.8% respectively. The confusion matrix and classwise accuracies for a threshold of 50, without STDP implementation, are shown in Fig. 6.5. While the overall accuracy drops to 59.41%, the network has learned meaningful discrimination between classes; ventricular classification increased to 38.4% TP rate and fusion classification increased to 28.9%. Supraventricular is still nearly undetectable, at 0.3%.

```
Confusion Matrix (rows=true, cols=predicted):
      N      S      V      F      Q
N  28171   254  12649   3145    26
S   1312     5    425    95     0
V   1955    13   1236    14     2
F    126     0    150   112     0
Q      6     0     1     0     0

Per-class accuracy:
N: 28171/44245 = 0.6367
S: 5/1837 = 0.0027
V: 1236/3220 = 0.3839
F: 112/388 = 0.2887
Q: 0/7 = 0.0000

Overall: 29524/49697 = 0.5941
```

Fig. 6.5: Improved Model without STDP - Confusion Matrix and Classwise Accuracy

Fig. 6.6 shows the confusion matrix and classwise accuracy for a threshold of 50, with STDP enabled. Along with the lower threshold values, the model collapses back to near-all normal classifications. This reflects that the current implementation of the STDP learning rule, in this capacity, is destructive to the supervised, learned weights during inference. The step sizes of the LUT and quantity of spikes across all test samples drifts the synaptic weights from the trained weight values; this causes the network to converge toward a single dominant neuron. This suggests that on-chip STDP would require smaller changes or clamping to constrain the drift from trained weights.

```

Confusion Matrix (rows=true, cols=predicted):
      N      S      V      F      Q
N  44202    38      4      1      0
S   1836      1      0      0      0
V   3220      0      0      0      0
F    388      0      0      0      0
Q      7      0      0      0      0

Per-class accuracy:
N: 44202/44245 = 0.9990
S: 1/1837 = 0.0005
V: 0/3220 = 0.0000
F: 0/388 = 0.0000
Q: 0/7 = 0.0000

Overall: 44203/49697 = 0.8895

```

Fig. 6.6: Improved Model with STDP - Confusion Matrix and Classwise Accuracy

Additionally, the confusion matrix reflects that different classes need different levels of sensitivity to be detected, promoting the testing of per-class thresholds. This would allow each class to accumulate the required amount of evidence to appropriately fire for its column and learned weights. From a hardware standpoint of this system, this would simply require passing in 4 additional threshold values via the scan-chain interface. Fig. 6.7 shows the confusion matrix and classwise accuracy for the same system with per-class thresholds. It reveals that all four main classes can simultaneously have some meaningful accuracy, and the per-neuron thresholds enabled the network to detect the 4 classes with sufficient samples. Tuning the per-class threshold values reveals that small changes in the threshold are extremely sensitive and create significant fluctuations in classwise accuracy. This shows that the weight structure has real discriminative information encoded in it – the trained weights genuinely capture the differences between the arrhythmia classes. However, the tight decision boundary suggests that with only 10 weights, the network can't create large enough separations between classes in terms of the membrane potential and ensuing action potentials.

Thresholds: N=12 S=10 V=9 F=150 Q=5

Confusion Matrix (rows=true, cols=predicted):

	N	S	V	F	Q
N	6587	9036	26466	2156	0
S	367	277	1122	71	0
V	91	859	2029	241	0
F	8	38	63	279	0
Q	0	3	4	0	0

Per-class accuracy:

N: 6587/44245 = 0.1489

S: 277/1837 = 0.1508

V: 2029/3220 = 0.6301

F: 279/388 = 0.7191

Q: 0/7 = 0.0000

Overall: 9172/49697 = 0.1846

Fig. 6.7: Improved Model with Per-Class Thresholds – Confusion Matrix and Classwise Accuracy

7. CONCLUSION

The proposed neuromorphic hardware system offers a highly efficient and scalable solution for real-time arrhythmia detection, bridging the gap between high-fidelity ECG monitoring and low-power edge computing for on-chip classification. Using a delta-modulator based spike encoder, a dot-product engine-based spiking neural network array, and winner-take all circuit, this architecture performs classifications while achieving significant energy savings through event-driven processing and localized memory. The integration of online learning via spike-timing-dependent-plasticity and a scan-chain wrapper to efficiently read and process system parameters ensures the system remains adaptable to diverse patient profiles through runtime reconfigurability, which also enables the processing of other biomedical signals aside from ECGs. Validated across four FPGA platforms, the design boasts a high maximum operating frequency, low power consumption that linearly scales with frequency, and an efficient logic footprint. ASIC implementation was done with a 130 nm CMOS process, which affirmed the FPGA findings – a compact silicon footprint, low power consumption, and strong timing performance. A software model was utilized to test the classification accuracy, through the use of a hardware-faithful simulation on weights trained using `snnTorch` built in functions; tested on the MIT-BIH Arrhythmia Database, the system showed both the potential and current limitations in classification across heartbeat classes, while identifying specific areas where improvements to the architecture – detailed in the following section – can further boost accuracy. These results demonstrate the potential of this neuromorphic approach for the next generation of battery powered intelligent wearable medical devices, featuring low-power execution for real-time, on-chip biomedical signal classification.

7.1. Future Work

While this initial design shows strong power, performance, and area results in ASIC implementation, the software model reveals areas for improvement and future work to improve the classification accuracy and move towards deployment in wearable devices. An overview detailing the potential future work, based on findings from these contributions and supporting literature, is as follows:

- Scaling the DPE Array/ Multi-layer SNNs [5], [61]: The software results revealed that the current 2 by 5, single-layer architecture, limited to 10 synapse weights, is insufficient to separate all heartbeat classes with wide margins. Cascading a second, 5 by 5 array might allow the first layer to detect low level features in the input and enable the second layer to combine those features into stronger class decisions. This method adds 25 synapses and 5 neurons – a modest area overhead – but preserves the event-driven architecture.
- Increasing Parameter Bit-Widths [61], [62]: The software evaluation revealed the transition from 4-bit synapse weights to 6-bits enabled better multi-class discrimination that was difficult at the lower precision level. An analysis of the area-accuracy tradeoffs when changing the precision of synapse weights and accumulated membrane potentials can potentially improve classification margins without significant area cost.
- Per-Neuron Programmable Thresholds [63], [64]: A key finding in the software model was that per-neuron thresholds enabled multi-class distinctions, and the tuning of these thresholds can dramatically affect the accuracy per class.

Implementing a programmable per neuron threshold, configurable through the current scan-chain interface, could improve the multi-class classification accuracy and allow the system to be tuned for specific clinical priorities. This could lead to the implementation of adaptive thresholds, or thresholds as a learnable parameter.

- Constrained STDP [65]: The current implementation of STDP proved to be destructive to the trained weights during inference, because the LUT step sizes are quite large relative to the magnitude of synapse weights. Implementing weight clamping in hardware – constraining the STDP updates to a certain window around the trained weights – would allow the system to perform online learning and remain adaptable to signal variations without drifting from its baseline classification ability. This may prompt the use of reward-modulated STDP (R-STDP), which combines standard, unsupervised STDP with reinforcement learning.
- Power-Frequency Scaling [66]: One of the most relevant takeaways from the FPGA and ASIC implementations is the extremely high maximum operating frequency of the architecture, paired with the linear dynamic power versus frequency relationship. The ECG signal is sampled at 360 Hz – system operation at even 1 MHz results in a 2,777 times speed advantage. The excess throughput can be traded for significant power savings through aggressive clock gating or voltage scaling between heartbeats. Alternatively, the idle cycles could be used to improve the classification and robustness without additional hardware, by processing multiple ECG leads or rerunning the classification with different parameter configurations. Dynamic voltage and frequency scaling (DVFS) is common in neuromorphic hardware and is directly applicable in this system.

- Expanding Input Dimensionality [67]: The current implementation encodes the up/down transitions of a single ECG channel into two spike stream inputs. Adding more input rows for the DPE array could capture more signal information; this can be achieved through several approaches. Amplitude slicing can utilize multiple delta modulators with varying step sizes to capture both coarse and fine changes in the signal. Temporal windowing can be used to segment the ECG wave temporally and each segment can pass to a different input. The use of multiple ECG leads can provide more information about cardiac activity. Simply down sampling the ECG signal can lead to higher performance as well.
- Evolutionary Optimization [68]: This method of training can map directly to this system; it requires a fitness function – classification accuracy on the hardware – to evaluate as opposed to any gradients. In this approach, populations of candidate weights and threshold configurations are evaluated directly on the hardware and iteratively refined, in a loop, through selection and mutation. The hardware-in-the-loop weight optimization, possibly through the scan-chain, would eliminate the software-to-hardware mismatch in training and could integrate well with the compactness of this design.

REFERENCES

- [1] J. Galvis-Chacón, O. Ramos-Soto, D. Oliva, A. Valdivia-G, H. Rostro-Gonzalez, and A. Patino-Saucedo, “Robust ECG signal classification using spiking neural networks with axonal delays,” *Neurocomputing*, vol. 667, Feb. 2026, doi: 10.1016/j.neucom.2025.132259.
- [2] S. Nattel *et al.*, “Early management of atrial fibrillation to prevent cardiovascular complications,” *Eur. Heart J.*, vol. 35, no. 22, pp. 1448–56, Jun. 2014, doi: 10.1093/eurheartj/ehu028.
- [3] D. Banerjee, S. Dey, and A. Pal, “An SNN Based ECG Classifier For Wearable Edge Devices,” Nov. 2022.
- [4] S. Gerber, M. Steiner, Maryada, G. Indiveri, and E. Donati, “Neuromorphic implementation of ECG anomaly detection using delay chains,” Sep. 2022, [Online]. Available: <http://arxiv.org/abs/2209.01266>
- [5] H. Chu *et al.*, “A Neuromorphic Processing System With Spike-Driven SNN Processor for Wearable ECG Classification,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 16, no. 4, pp. 511–523, Aug. 2022, doi: 10.1109/TBCAS.2022.3189364.
- [6] C. Enoch and G. Promise, “Artificial Intelligence in Electrocardiogram (ECG) Analysis,” Aug. 2025.
- [7] F. C. Bauer, D. R. Muir, and G. Indiveri, “Real-Time Ultra-Low Power ECG Anomaly Detection Using an Event-Driven Neuromorphic Processor,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 6, pp. 1575–1582, Dec. 2019, doi: 10.1109/TBCAS.2019.2953001.

- [8] F. Ponulak and A. Kasinski, "Introduction to spiking neural networks: Information processing, learning and applications.," *Acta Neurobiol. Exp. (Wars)*., vol. 71, no. 4, pp. 409–33, 2011, doi: 10.55782/ane-2011-1862.
- [9] G. Indiveri and S. C. Liu, "Memory and Information Processing in Neuromorphic Systems," Aug. 01, 2015, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/JPROC.2015.2444094.
- [10] "Cardiovascular diseases (CVDs)," World Health Organization.
- [11] "Cardiac electrophysiology and ECG interpretation – The Cardiovascular." Accessed: May 08, 2026. [Online]. Available: <https://ecgwaves.com/topic/introduction-electrocardiography-ecg-book/>
- [12] "Mechanisms of cardiac arrhythmias: from automaticity to re-entry (reentry) – The Cardiovascular." Accessed: May 08, 2026. [Online]. Available: <https://ecgwaves.com/topic/mechanisms-cardiac-arrhythmias-automaticity-reentry-triggered-activity/>
- [13] "ECG interpretation: Characteristics of the normal ECG (P-wave, QRS complex, ST segment, T-wave) – The Cardiovascular." Accessed: May 08, 2026. [Online]. Available: <https://ecgwaves.com/topic/ecg-normal-p-wave-qrs-complex-st-segment-t-wave-j-point/>
- [14] "Understanding Arrhythmias and Heart Rhythm Disorders | Knight Cardiovascular Institute | OHSU." Accessed: May 08, 2026. [Online]. Available: <https://www.ohsu.edu/knight-cardiovascular-institute/understanding-arrhythmias-and-heart-rhythm-disorders>

- [15] S.-K. Chua *et al.*, “Comparison of Arrhythmia Detection by 24-Hour Holter and 14-Day Continuous Electrocardiography Patch Monitoring,” *Acta Cardiol. Sin.*, vol. 36, no. 3, pp. 251–259, May 2020, doi: 10.6515/ACS.202005_36(3).20190903A.
- [16] “Recognized Consensus Standards: Medical Devices.” Accessed: May 15, 2026. [Online]. Available: https://www.accessdata.fda.gov/scripts/cdrh/cfdocs/cfstandards/detail.cfm?standard__identification_no=31679
- [17] G. B. Moody and R. G. Mark, “The impact of the MIT-BIH arrhythmia database,” *IEEE Eng. Med. Biol. Mag.*, vol. 20, no. 3, pp. 45–50, 2001, doi: 10.1109/51.932724.
- [18] J. Guan, W. Wang, P. Feng, X. Wang, and W. Wang, “Low-Dimensional Denoising Embedding Transformer for ECG Classification,” in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, Jun. 2021, pp. 1285–1289. doi: 10.1109/ICASSP39728.2021.9413766.
- [19] M. S. Patel, D. A. Asch, and K. G. Volpp, “Wearable devices as facilitators, not drivers, of health behavior change,” *JAMA*, vol. 313, no. 5, pp. 459–60, Feb. 2015, doi: 10.1001/jama.2014.14781.
- [20] V. A. Odeh, Y. Chen, W. Wang, and X. Ding, “Recent Advances in the Wearable Devices for Monitoring and Management of Heart Failure,” *Rev. Cardiovasc. Med.*, vol. 25, no. 10, p. 386, Oct. 2024, doi: 10.31083/j.rcm2510386.

- [21] J. S. Steinberg *et al.*, “2017 ISHNE-HRS expert consensus statement on ambulatory ECG and external cardiac monitoring/telemetry.,” *Ann. Noninvasive Electrocardiol.*, vol. 22, no. 3, May 2017, doi: 10.1111/anec.12447.
- [22] M. T. Jensen *et al.*, “ESC working group on e-cardiology position paper: use of commercially available wearable technology for heart rate and activity tracking in primary and secondary cardiovascular prevention-in collaboration with the European Heart Rhythm Association, European Association of Preventive Cardiology, Association of Cardiovascular Nursing and Allied Professionals, Patient Forum, and the Digital Health Committee.,” *European heart journal. Digital health*, vol. 2, no. 1, pp. 49–59, Mar. 2021, doi: 10.1093/ehjdh/ztab011.
- [23] M. A. ROSENBERG, M. SAMUEL, A. THOSANI, and P. J. ZIMETBAUM, “Use of a Noninvasive Continuous Monitoring Device in the Management of Atrial Fibrillation: A Pilot Study,” *Pacing and Clinical Electrophysiology*, vol. 36, no. 3, pp. 328–333, Mar. 2013, doi: 10.1111/pace.12053.
- [24] M. R. Cowie *et al.*, “Improving care for patients with acute heart failure: before, during and after hospitalization.,” *ESC Heart Fail.*, vol. 1, no. 2, pp. 110–145, Dec. 2014, doi: 10.1002/ehf2.12021.
- [25] H. Narotamo, M. Dias, R. Santos, A. V. Carreiro, H. Gamboa, and M. Silveira, “Deep learning for ECG classification: A comparative study of 1D and 2D representations and multimodal fusion approaches,” *Biomed. Signal Process. Control*, vol. 93, p. 106141, Jul. 2024, doi: 10.1016/j.bspc.2024.106141.

- [26] Z. Ahmad, A. Tabassum, L. Guan, and N. M. Khan, "ECG Heartbeat Classification Using Multimodal Fusion," *IEEE Access*, vol. 9, pp. 100615–100626, 2021, doi: 10.1109/ACCESS.2021.3097614.
- [27] M. Ben Ammar, I. Ben Dhaou, D. El Houssaini, S. Sahnoun, A. Fakhfakh, and O. Kanoun, "Requirements for Energy-Harvesting-Driven Edge Devices Using Task-Offloading Approaches," *Electronics (Basel)*, vol. 11, no. 3, p. 383, Jan. 2022, doi: 10.3390/electronics11030383.
- [28] L. Deng *et al.*, "Rethinking the performance comparison between SNNS and ANNS," *Neural Networks*, vol. 121, pp. 294–307, Jan. 2020, doi: 10.1016/j.neunet.2019.09.005.
- [29] M. Dampfhofer, T. Mesquida, A. Valentian, and L. Anghel, "Are SNNs Really More Energy-Efficient Than ANNs? an In-Depth Hardware-Aware Study," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 7, no. 3, pp. 731–741, Jun. 2023, doi: 10.1109/TETCI.2022.3214509.
- [30] Wulfram. Gerstner, W. M. . Kistler, Richard. Naud, and Liam. Paninski, *Neuronal dynamics : from single neurons to networks and models of cognition*. Cambridge University Press, 2015.
- [31] Peter. Dayan and Laurence. Abbott, *Theoretical Neuroscience : Computational and Mathematical Modeling of Neural Systems*. MIT Press, 2014.
- [32] N. N. Chakraborty, G. S. Rose, and M. H. Kao, "Programmable Refractory Period Implementations in a Mixed-Signal Integrate-And-Fire Neuron," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, IEEE, May 2022, pp. 770–774. doi: 10.1109/ISCAS48785.2022.9938009.

- [33] H. Dong, J. Li, B. Lan, M. Xu, and L. Chen, “Hardware implementation of SNN-Based,” in *Proceedings of the 2024 2nd Asia Symposium on Image and Graphics*, New York, NY, USA: ACM, Dec. 2024, pp. 67–72. doi: 10.1145/3718441.3718451.
- [34] I. Semenov and D. Nikitin, “Advantages and Disadvantages of Spiking Neural Networks Compared to Classical Artificial Neural Networks,” in *2023 Seminar on Information Computing and Processing (ICP)*, IEEE, Nov. 2023, pp. 192–194. doi: 10.1109/ICP60417.2023.10397433.
- [35] J. K. Eshraghian *et al.*, “Training Spiking Neural Networks Using Lessons From Deep Learning,” *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, Sep. 2023, doi: 10.1109/JPROC.2023.3308088.
- [36] W. Guo, M. E. Fouda, A. M. Eltawil, and K. N. Salama, “Neural Coding in Spiking Neural Networks: A Comparative Study for Robust Neuromorphic Systems,” *Front. Neurosci.*, vol. 15, p. 638474, 2021, doi: 10.3389/fnins.2021.638474.
- [37] S. H. Choi, “Spiking neural networks for biomedical signal analysis,” *Biomed. Eng. Lett.*, vol. 14, no. 5, pp. 955–966, Sep. 2024, doi: 10.1007/s13534-024-00405-z.
- [38] B. Gardner and A. Grüning, “Supervised Learning in Spiking Neural Networks for Precise Temporal Encoding,” Oct. 2016, doi: 10.1371/journal.pone.0161335.
- [39] F. Corradi *et al.*, “ECG-based Heartbeat Classification in Neuromorphic Hardware,” in *2019 International Joint Conference on Neural Networks (IJCNN)*, IEEE, Jul. 2019, pp. 1–8. doi: 10.1109/IJCNN.2019.8852279.

- [40] M. Hu *et al.*, “Dot-product engine for neuromorphic computing,” in *Proceedings of the 53rd Annual Design Automation Conference*, New York, NY, USA: ACM, Jun. 2016, pp. 1–6. doi: 10.1145/2897937.2898010.
- [41] “How the von Neumann bottleneck is impeding AI computing - IBM Research.” Accessed: May 08, 2026. [Online]. Available: <https://research.ibm.com/blog/why-von-neumann-architecture-is-impeding-the-power-of-ai-computing>
- [42] D. Joksas *et al.*, “Memristive, Spintronic, and 2D-Materials-Based Devices to Improve and Complement Computing Hardware,” *Advanced Intelligent Systems*, vol. 4, no. 8, Aug. 2022, doi: 10.1002/aisy.202200068.
- [43] E. Jahns, D. Moreno, and M. A. Kinsy, “Learning Neuron Dynamics within Deep Spiking Neural Networks,” Oct. 2025, [Online]. Available: <http://arxiv.org/abs/2510.07341>
- [44] N. A. Rahman and N. Yusoff, “Modulated spike-time dependent plasticity (STDP)-based learning for spiking neural network (SNN): A review,” *Neurocomputing*, vol. 618, p. 129170, Feb. 2025, doi: 10.1016/j.neucom.2024.129170.
- [45] P. Ferré, F. Mamalet, and S. J. Thorpe, “Unsupervised Feature Learning With Winner-Takes-All Based STDP,” *Front. Comput. Neurosci.*, vol. 12, Apr. 2018, doi: 10.3389/fncom.2018.00024.
- [46] N. N. Chakraborty, H. Das, and G. S. Rose, “Spike-Timing-Dependent Plasticity for a Hafnium-Oxide Memristive Synapse,” in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, IEEE, Aug. 2023, pp. 463–467. doi: 10.1109/MWSCAS57524.2023.10406099.

- [47] F. Liu, W. Zhao, Y. Chen, Z. Wang, T. Yang, and L. Jiang, “SSTDTP: Supervised Spike Timing Dependent Plasticity for Efficient Spiking Neural Network Training,” *Front. Neurosci.*, vol. 15, Nov. 2021, doi: 10.3389/fnins.2021.756876.
- [48] X. Liu, “Inhibition SNN: unveiling the efficacy of various lateral inhibition learning in image pattern recognition,” *Discover Applied Sciences*, vol. 6, no. 11, p. 611, Nov. 2024, doi: 10.1007/s42452-024-06332-z.
- [49] Xilinx and Inc, “7 Series FPGAs Data Sheet: Overview (DS180),” 2010. [Online]. Available: www.xilinx.com
- [50] “OpenLane Documentation.” Accessed: May 11, 2026. [Online]. Available: <https://openlane2.readthedocs.io/en/latest/>
- [51] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-based Tapeout- Proven Flow : Invited Paper,” in *IEEE/ACM International Conference on Computer-Aided Design, Digest of Technical Papers, ICCAD*, Institute of Electrical and Electronics Engineers Inc., Nov. 2020. doi: 10.1145/3400302.3415735.
- [52] “google/skywater-pdk: Open source process design kit for usage with SkyWater Technology Foundry’s 130nm node.” Accessed: May 11, 2026. [Online]. Available: <https://github.com/google/skywater-pdk>
- [53] “Configuration Variables - OpenLane Documentation.” Accessed: May 11, 2026. [Online]. Available: <https://armleo-openlane.readthedocs.io/en/latest/docs/source/configuration.html>
- [54] “Introduction — snntorch 0.9.4 documentation.” Accessed: May 14, 2026. [Online]. Available: <https://snntorch.readthedocs.io/en/latest/readme.html>

- [55] “snn.Leaky — snntorch 0.9.4 documentation.” Accessed: May 14, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/snn.neurons_leaky.html
- [56] “Tutorial 5 - Training Spiking Neural Networks with snntorch — snntorch 0.9.4 documentation.” Accessed: May 14, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_5.html
- [57] E. O. Neftci, H. Mostafa, and F. Zenke, “Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks,” *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, Nov. 2019, doi: 10.1109/MSP.2019.2931595.
- [58] “Tutorial 6 - Surrogate Gradient Descent in a Convolutional SNN — snntorch 0.9.4 documentation.” Accessed: May 16, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_6.html
- [59] “snntorch.functional — snntorch 0.9.4 documentation.” Accessed: May 15, 2026. [Online]. Available: <https://snntorch.readthedocs.io/en/latest/snntorch.functional.html>
- [60] P. de Chazal, M. O’Dwyer, and R. B. Reilly, “Automatic classification of heartbeats using ECG morphology and heartbeat interval features,” *IEEE Trans. Biomed. Eng.*, vol. 51, no. 7, pp. 1196–206, Jul. 2004, doi: 10.1109/TBME.2004.827359.
- [61] N. Rathi *et al.*, “Exploring Neuromorphic Computing Based on Spiking Neural Networks: Algorithms to Hardware,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–49, Dec. 2023, doi: 10.1145/3571155.

- [62] H. W. Lui and E. Neftci, “Hessian Aware Quantization of Spiking Neural Networks,” in *International Conference on Neuromorphic Systems 2021*, New York, NY, USA: ACM, Jul. 2021, pp. 1–5. doi: 10.1145/3477145.3477158.
- [63] A. Shaban, S. S. Bezugam, and M. Suri, “An adaptive threshold neuron for recurrent spiking neural networks with nanodevice hardware implementation,” *Nat. Commun.*, vol. 12, no. 1, p. 4234, Jul. 2021, doi: 10.1038/s41467-021-24427-8.
- [64] Y. Wang, Y. Xu, R. Yan, and H. Tang, “Deep Spiking Neural Networks With Binary Weights for Object Recognition,” *IEEE Trans. Cogn. Dev. Syst.*, vol. 13, no. 3, pp. 514–523, Sep. 2021, doi: 10.1109/TCDS.2020.2971655.
- [65] A. Amirshahi and M. Hashemi, “ECG Classification Algorithm Based on STDP and R-STDP Neural Networks for Real-Time Monitoring on Ultra Low-Power Personal Wearable Devices,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 6, pp. 1483–1493, Dec. 2019, doi: 10.1109/TBCAS.2019.2948920.
- [66] S. Hoppner *et al.*, “Dynamic Power Management for Neuromorphic Many-Core Systems,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 8, pp. 2973–2986, Aug. 2019, doi: 10.1109/TCSI.2019.2911898.
- [67] B.-J. Singstad and E. M. Muten, “Assessing the Impact of Downsampled ECGs and Alternative Loss Functions in Multi-Label Classification of 12-Lead ECGs,” *Cardiovasc. Eng. Technol.*, vol. 16, no. 5, pp. 599–610, Oct. 2025, doi: 10.1007/s13239-025-00797-8.

- [68] C. D. Schuman, J. P. Mitchell, R. M. Patton, T. E. Potok, and J. S. Plank, “Evolutionary Optimization for Neuromorphic Systems,” in *Proceedings of the Neuro-inspired Computational Elements Workshop*, New York, NY, USA: ACM, Mar. 2020, pp. 1–9. doi: 10.1145/3381755.3381758.

APPENDIX

The following SystemVerilog modules comprise the complete RTL implementation of the neuromorphic ECG classifier.

A.1 – Delta Modulator

```
module delta_mod#(
    parameter DATA_WIDTH = 11,
    parameter MAX_VAL = 2047,
    parameter MIN_VAL = 0
)(
    input logic clk,
    input logic dm_reset,
    input logic [3:0] STEP_SIZE, //threshold for spiking
    input logic [DATA_WIDTH-1:0] ecg_in,
    output logic [1:0] dm_spike_out,          //[0] UP, [1] DOWN
    output logic [DATA_WIDTH-1:0] signal
);

    logic init;
    logic last_cycle_spiked;

    wire [DATA_WIDTH-1:0] next_up = (signal + STEP_SIZE > MAX_VAL) ? MAX_VAL : (signal + STEP_SIZE);
    wire [DATA_WIDTH-1:0] next_down = (signal < STEP_SIZE) ? MIN_VAL : (signal - STEP_SIZE);

    always_ff @(posedge clk or posedge dm_reset) begin
        if (dm_reset) begin
            signal <= '0;
            dm_spike_out[0] <= 1'b0;
            dm_spike_out[1] <= 1'b0;
            init <= 1'b0;
            last_cycle_spiked <= 1'b0;
        end
    end
endmodule
```



```

end else begin

    dm_spike_out[0] <= 1'b0;

    dm_spike_out[1] <= 1'b0;

    if (!init) begin

        signal <= ecg_in;

        init <= 1'b1;

    end else if (last_cycle_spiked) begin

        last_cycle_spiked <= 1'b0;

    end else begin

        if (ecg_in > signal + STEP_SIZE) begin

            dm_spike_out[0] <= 1'b1;

            last_cycle_spiked <= 1'b1;

            signal <= next_up;

        end else if (ecg_in + STEP_SIZE < signal) begin

            dm_spike_out[1] <= 1'b1;

            last_cycle_spiked <= 1'b1;

            signal <= next_down;

        end

    end

end

end

end

endmodule

```

A.2 – Synapse Unit

```
module synapse_unit_2#(
    parameter int b_weight = 4,
    parameter int b_time = 3,
    parameter int LEARN = 3
) (
    input logic clk,
    input logic reset,
    input logic load_config,
    input logic enable,
    input logic stdp_enable,
    input logic signed spike_in,          // 1 or 0 for UP or DOWN
    input logic [b_time:0] neuron_time,    // Time since neuron last fired - cycles
    input logic signed [b_weight+1:0] init_weight,
    output logic signed [b_weight+1:0] weight_to_accum // To accumulator (spike x weight)
);

//Reg for weight and syn tim logic
logic signed [b_weight+1:0] weight_reg;
logic [b_time:0] syn_time;
logic signed [b_time:0] delta_t;          //Calculate delta-t post-pre
logic stdp_calc;                          //STDP logic- enabled, spike, and within window
logic [b_time:0] lut_index;
logic [b_time:0] lut_index_reg;
logic stdp_calc_reg;

typedef logic signed [3:0] lut_t;
localparam lut_t stdp_lut [0:6] = '{1,2,3,0,-2,-1,0};

always_comb begin
    delta_t = $signed({1'b0, neuron_time}) - $signed({1'b0, syn_time});

    if (((delta_t < -LEARN) || (delta_t > LEARN))) begin
        lut_index = LEARN;
        stdp_calc = 1'b0;
    end else begin
        if (neuron_time == 0 && spike_in != 0)
            lut_index = LEARN + 1;
        else
            lut_index = delta_t + LEARN;

        stdp_calc = stdp_enable && (((spike_in != 0) && (neuron_time < LEARN)) ||
(neuron_time == 0));
```

```

        end
    end

    always_ff @(posedge clk) begin
        if (reset) begin
            stdp_calc_reg <= '0;
            lut_index_reg <= LEARN;
        end else begin
            stdp_calc_reg <= stdp_calc;
            lut_index_reg <= lut_index;
        end
    end

    end

    //assign updated weight to reg if stdp logic
    always_ff @(posedge clk) begin
        if (reset) begin
            weight_reg <= init_weight;
            syn_time <= 4'd7;
        end else if (load_config) begin
            weight_reg <= init_weight;
            syn_time <= 4'd7;
        end else if (enable) begin
            if (spike_in != 0)
                syn_time <= 0;
            else if (syn_time < 7)
                syn_time <= syn_time + 1'b1;
            if (stdp_calc_reg) begin
                logic signed [b_weight+1:0] next_w;
                next_w = weight_reg + stdp_lut[lut_index_reg];
                if (next_w == 0)
                    weight_reg <= 1'b1;
                else
                    weight_reg <= next_w;
            end
        end
    end

    end

    //assign time output and product of weight and spike to be accum
    assign weight_to_accum = (enable && spike_in != 0) ?
        ($signed(weight_reg)) : '0;

endmodule

```

A.3 – Accumulator

```
module accumulator #(
    parameter int N_inputs = 2,
    parameter int b_weight = 4, // Width of each synapse input
    parameter int b_accum = 8,  // Width of accumulator (membrane potential)
    parameter int b_time = 3,
    parameter int b_thresh = 8
) (
    input logic clk,
    input logic reset,
    input logic enable,
    input logic signed [b_weight+1:0] syn_in [N_inputs],
    input logic [b_time:0] refractory, //refractory period in clock cycles
    input logic [b_time:0] leak_rate,  //-1 every leak_rate clock cycles
    input logic [b_thresh-1:0] threshold,
    input logic [b_time:0] neuron_time, // Time since neuron last fired - cycles
    input logic any_fire,
    output logic fire, // Reset trigger
    output logic signed [b_accum+1:0] accum_out // Membrane potential
);

(* max_fanout = 4 *) logic signed [b_accum+1:0] accum_reg;
logic [b_time:0] leak_counter;
(* max_fanout = 4 *) logic signed [b_accum+1:0] pre_accum;
logic accum_on;

//preaccum
always_ff @(posedge clk) begin
    if (reset)
        pre_accum <= '0;
    else begin
        automatic logic signed [b_accum+1:0] sum = '0;
        for (int i = 0; i < N_inputs; i++)
            sum = sum + $signed(syn_in[i]);
        pre_accum <= sum;
    end
end

//compare and fire
always_ff @(posedge clk) begin
    //pre_accum <= $signed(syn_0) + $signed(syn_1);
```

```

        if (fire)
            fire <= 1'b0;
        else
            fire <= enable && (accum_reg >= $signed(threshold));
        end

    assign accum_on = enable && (neuron_time > refractory - 1'b1);

    always_ff @(posedge clk) begin
        if (reset) begin
            accum_reg <= 0;
            leak_counter <= '0;
        end else if (fire || any_fire) begin
            accum_reg <= 0;
            leak_counter <= '0;
        end else if (accum_on) begin
            automatic logic signed [b_accum+1:0] next_accum = accum_reg + pre_accum;

            if (leak_counter < leak_rate) begin
                accum_reg <= next_accum;
                leak_counter <= leak_counter + 1'b1;
            end else begin
                leak_counter <= '0;
                accum_reg <= (next_accum > 0) ? (next_accum - 1'b1) :
                    (next_accum < 0) ? (next_accum + 1'b1) :
                        next_accum;
            end
        end
    end

    assign accum_out = accum_reg;

endmodule

```

A.4 – WTA

```
module wta#(  
    parameter int N_neurons = 5  
)(  
    input logic enable,  
    input logic [N_neurons-1:0] fire,  
    output logic [N_neurons-1:0] winner,  
    output logic [N_neurons-1:0] reset_accums  
);  
  
    logic any_fire;  
  
    always_comb begin  
        any_fire = '0;  
        winner = '0;  
        reset_accums = '0;  
  
        if (enable) begin  
            any_fire = |fire;  
            for (int i = 0; i < N_neurons; i++) begin  
                if (fire[i]) begin  
                    winner[i] = 1'b1;  
                    break;  
                end  
            end  
  
            reset_accums = any_fire ? (~winner) : '0;  
        end  
    end  
  
endmodule
```

A.5 – Neuron Array

```
module neuron_array#(
    parameter int N_inputs = 2,
    parameter int N_neurons = 5,
    parameter int b_weight = 4,
    parameter int b_accum = 8,
    parameter int b_thresh = 8,
    parameter int b_time = 3,
    parameter int LEARN = 3
) (
    input logic clk,
    input logic reset,
    input logic load_config, //SCW
    input logic [N_inputs-1:0] spike_in, //N_inputs rows of inputs
    input logic signed [b_weight+1:0] synapse_weights [N_inputs*N_neurons], //SCW
    input logic [b_thresh-1:0] threshold, //SCW
    input logic [b_time:0] refractory, //SCW
    input logic [b_time:0] leak_rate, //SCW
    output logic [N_neurons-1:0] fire, //N_neurons columns of outputs
    output logic [N_neurons-1:0] winner
);

//neuron time for each row of synapses
//TIMING (FOR STDP)
logic [b_time:0] neuron_time [N_neurons];

always_ff @(posedge clk or posedge reset) begin : post_timing
    if (reset) begin
        for (int j = 0; j < N_neurons; j++)
            neuron_time[j] <= {b_time{1'b1}};
        end
    else begin
        for (int j = 0; j < N_neurons; j++) begin
            if (fire[j] != 0)
                neuron_time[j] <= '0;
            else if (neuron_time[j] < {b_time{1'b1}})
                neuron_time[j] <= neuron_time[j] + 3'd1;
            end
        end
    end
end
```

```

//ENABLE LOGIC
logic sys_en, load_active;

always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        load_active <= 1'b0;
        sys_en <= 1'b0;
    end else begin
        load_active <= load_config;
        if (load_active)
            sys_en <= 1'b1;
    end
end

//synapse outputs, one per input to each output
//N-inputs*N_neurons synapse_units, weight_to_accum singals
logic signed [b_weight+1:0] syn_out [N_inputs*N_neurons];

genvar i, j;
generate
    for (i = 0; i < N_inputs; i++) begin : gen_inputs
        for (j = 0; j < N_neurons; j++) begin : gen_outputs
            synapse_unit_2 #(
                .b_weight(b_weight),
                .b_time(b_time),
                .LEARN(LEARN)
            ) syn_inst (
                .clk(clk),
                .reset(reset),
                .load_config(load_active),
                .enable(sys_en),
                .stdp_enable(1'b1),
                .spike_in(spike_in[i]),
                .neuron_time(neuron_time[j]),
                .init_weight(synapse_weights[N_neurons*i+j]),
                .weight_to_accum(syn_out[N_neurons*i+j])
            );
        end
    end
endgenerate

logic signed [b_accum+1:0] accum_cols [N_neurons];

```



```

genvar k;
generate
    for (k = 0; k < N_neurons; k++) begin : gen_cols
        logic signed [b_weight+1:0] syn_col [N_inputs];
        for (genvar r = 0; r < N_inputs; r++) begin : gen_syn_map
            assign syn_col[r] = syn_out[N_neurons*r + k];
        end

        accumulator #(
            .N_inputs(N_inputs),
            .b_weight(b_weight),
            .b_accum(b_accum),
            .b_time(b_time),
            .b_thresh(b_thresh)
        ) acc_inst (
            .clk(clk),
            .reset(reset),
            .enable(sys_en),
            .syn_in(syn_col),
            .refractory(refractory),
            .leak_rate(leak_rate),
            .threshold(threshold),
            .neuron_time(neuron_time[k]),
            .any_fire(reset_accums[k]),
            .fire(fire[k]),
            .accum_out(accum_cols[k])
        );
    end
endgenerate

logic [N_neurons-1:0] reset_accums;
wta #(
    .N_neurons(N_neurons)
) wta_inst (
    .enable(sys_en),
    .fire(fire),
    .winner(winner),
    .reset_accums(reset_accums)
);

endmodule

```

A.6 – Scan Chain Wrapper

```
module scan_chain_wrapper#(
    parameter int N_inputs = 2,
    parameter int N_neurons = 5,
    parameter int b_weight = 4,
    parameter int b_accum = 8,
    parameter int b_thresh = 8,
    parameter int b_dm_step = 4,
    parameter int b_time = 3,
    parameter int LEARN = 3,
    parameter int DATA_WIDTH = 11,
    parameter int TOTAL_BITS = (N_inputs*N_neurons*(b_weight+2)) + (b_thresh) +
                                2*(b_time+1) + b_dm_step
) (
    //neuron_array
    input logic clk,
    input logic reset,

    //scw signals
    input logic prog_clk,
    input logic prog_data,
    input logic load_config,

    //DM
    input logic dm_reset,
    input logic [DATA_WIDTH-1:0] ecg_in,

    //neuron_array
    output logic [N_neurons-1:0] fire,
    output logic [N_neurons-1:0] winner
);

    logic [TOTAL_BITS-1:0] shift_reg;
    logic [TOTAL_BITS-1:0] config_reg;

    always_ff @(posedge prog_clk or posedge reset) begin
        if (reset)
            shift_reg <= '0;
        else
            shift_reg <= {shift_reg[TOTAL_BITS-2:0], prog_data};
    end
end
```

```

always_ff @(posedge clk or posedge reset) begin
    if (reset)
        config_reg <= '0;
    else if (load_config)
        config_reg <= shift_reg;
end

//when scanning go MSB to LSB
//start with weight[15][MSB->LSB]..weight[0][0]..config params MSB to LSB

localparam B_WEIGHT = b_weight+2;
localparam B_THRESH = b_thresh;
localparam B_REFRAC = b_time+1;
localparam B_LEAK = b_time+1;
localparam B_DM_STEP = b_dm_step;
localparam N = N_inputs*N_neurons;

//Scan chain inputs
logic signed [b_weight+1:0] synapse_weights [N];
logic [b_thresh-1:0] threshold;
logic [b_time:0] refractory;
logic [b_time:0] leak_rate;
logic [b_dm_step-1:0] dm_threshold;
int offset;

always_comb begin
    offset = TOTAL_BITS-1;

    //MSB N*6 bits for syn_weights
    for (int i = 0; i < N; i++) begin
        synapse_weights[N-1-i] = config_reg[offset -: B_WEIGHT];
        offset -= B_WEIGHT;
    end

    //rest is config
    threshold = config_reg[offset -: B_THRESH];
    offset -= B_THRESH;
    refractory = config_reg[offset -: B_REFRAC];
    offset -= B_REFRAC;
    leak_rate = config_reg[offset -: B_LEAK];
    offset -= B_LEAK;
    dm_threshold = config_reg[offset -: B_DM_STEP];
    offset -= B_DM_STEP;

```

```

end

//Instantiate
//DM DUT
logic [DATA_WIDTH-1:0] signal;
logic [1:0] dm_out;

delta_mod dm_dut(
    .clk(clk),
    .dm_reset(dm_reset),
    .STEP_SIZE(dm_threshold),
    .ecg_in(ecg_in),
    .dm_spike_out(dm_out),
    .signal(signal)
);

//Neuron Array DUT
neuron_array #(
    .N_inputs(N_inputs),
    .N_neurons(N_neurons),
    .b_weight(b_weight), //6
    .b_accum(b_accum),
    .b_thresh(b_thresh),
    .b_time(b_time),
    .LEARN(LEARN)
) dut (
    .clk(clk),
    .reset(reset),
    .load_config(load_config),
    .spike_in(dm_out),
    .synapse_weights(synapse_weights),
    .threshold(threshold),
    .refractory(refractory),
    .leak_rate(leak_rate),
    .fire(fire),
    .winner(winner)
);

endmodule

```